TECHNICAL BLUEPRINT · VERSION 1.0

ResolveHQ

Software Architecture & Technical Implementation Blueprint

A project management and issue-tracking SaaS platform for small development teams, freelancers and software agencies. Modular monolith on React, Node.js, Express and MongoDB, with role-based access control and an automated test suite.

PREPARED FOR	Bikendra Giri
DATE	November 3, 2026
DOCUMENT TYPE	Planning and implementation blueprint
STACK	React + Vite, Tailwind CSS, Node.js + Express, MongoDB + Mongoose, JWT, Vitest + Supertest
STATUS	All functionality is planned. Nothing described here has been implemented, deployed or used by clients.

Table of Contents

1. Project Overview . . . 5

1.1 Problem statement . . . 5

1.2 Target users and personas . . . 5

1.3 Domain glossary . . . 5

1.4 Design principles . . . 5

2. Technology Stack . . . 6

2.1 Core stack . . . 6

2.2 Supporting libraries (additions) . . . 6

2.3 Why a modular monolith . . . 6

3. Core Product Features . . . 7

3.11 MVP scope and optional enhancements . . . 8

4. System Architecture and Request Handling . . . 9

4.1 High-level architecture . . . 9

4.2 Layer responsibilities . . . 9

4.3 Backend project structure . . . 10

4.4 Express middleware execution order . . . 10

4.5 Browser request lifecycle: moving a Kanban card . . . 11

4.6 Separating business logic from HTTP handling . . . 12

4.7 Frontend request handling . . . 12

4.8 Environment variables and configuration . . . 13

5. Database and Data Relationships . . . 14

5.1 Entity relationship diagram . . . 14

5.2 Collection definitions . . . 14

5.3 Relationships and how MongoDB represents them . . . 18

5.4 Referential integrity . . . 18

5.5 Deletion strategies . . . 19

5.6 Data ownership and tenant isolation . . . 19

5.7 Query performance and indexing . . . 19

6. Authentication and Authorization . . . 20

6.1 Design summary . . . 20

6.2 Registration, login, refresh and logout flow . . . 20

6.3 Registration and password handling . . . 20

6.4 JWT creation, verification and expiration . . . 21

6.5 Token storage and handling . . . 21

6.6 Logout and token invalidation . . . 21

6.7 Backend authentication middleware . . . 22

6.8 Protected frontend routes . . . 22

6.9 Account and session security . . . 22
6.10 Role-based access control (RBAC) . . . 22
6.11 Permission matrix . . . 23
6.12 Enforcing authorization in code . . . 24

7. Validation, Security and Error Handling . . . 25

7.1 Input validation and sanitization . . . 25
7.2 Password and credential protection . . . 25
7.3 CORS, headers, rate limiting and brute-force protection . . . 25
7.4 NoSQL injection and unsafe input . . . 25
7.5 Environment variables and secrets . . . 26
7.6 Ownership checks and IDOR prevention . . . 26
7.7 Centralized error handling . . . 26
7.8 Realistic error examples . . . 27
7.9 Logging without leaking secrets . . . 27
7.10 Security checklist . . . 28
7.11 Principal security risks and trade-offs . . . 28

8. REST API Design . . . 29

8.1 Conventions . . . 29
8.2 Endpoint reference . . . 29
8.3 Request and response examples . . . 32

9. Activity Logs and Notifications . . . 35

9.1 How events flow . . . 35
9.2 Events that must be recorded . . . 35
9.3 Activity record . . . 35
9.4 Protecting logs from modification . . . 35
9.5 Notification creation rules . . . 36
9.6 Unread counts and read status . . . 36
9.7 Failure handling and its trade-off . . . 36

10. Testing and Quality Assurance . . . 37

10.1 Strategy . . . 37
10.2 Test environment . . . 37
10.3 Test matrix . . . 37
10.4 Running the tests . . . 39
10.5 Gate before deployment . . . 39

11. Development Roadmap . . . 40

12. Deployment and Maintenance . . . 41

12.1 Deployment topology . . . 41
12.2 Frontend deployment . . . 41
12.3 Backend deployment . . . 41

12.4 MongoDB hosting . . . 41
12.5 Environment configuration . . . 42
12.6 Production security . . . 42
12.7 Logging, monitoring and error tracking . . . 42
12.8 Backups and recovery . . . 42
12.9 Maintenance routine . . . 42
12.10 Deployment checklist . . . 43
12.11 Git and GitHub workflow . . . 43

13. Technical Decisions, Assumptions and Status . . . 44

13.1 Technical decisions and their reasoning . . . 44
13.2 Assumptions . . . 44
13.3 Planned versus implemented . . . 45
13.4 Open questions to settle early . . . 45
Appendix A. First setup commands (Phase 1) . . . 45

How to use this document. Read Sections 1 to 4 to understand what is being built and how requests flow. Sections 5 to 9 are the detailed design (data, security, API, history). Sections 10 to 12 explain how to test, build in phases and deploy. Section 13 records every decision, assumption and the planned-versus-implemented status. Diagrams are drawn directly in this PDF.

1. Project Overview

ResolveHQ is a project management and issue-tracking SaaS platform inspired by Jira and Linear. It is designed for small development teams, freelancers and software agencies who need one place for planning, task assignment, bug tracking, collaboration, progress monitoring and accountability.

This document is an implementation-ready blueprint: it defines what to build, how each part works, and in what order to build it. It is written so that one developer can follow it from an empty repository to a deployed MVP.

> **Status of this document.** Everything described here is **planned functionality**. No feature has been implemented, deployed or tested yet, and ResolveHQ has no users or clients. Statements such as "the API returns" describe the intended design. Where a choice is uncertain, the assumption is stated explicitly (see Section 13).

1.1 Problem statement

Small software teams commonly track work across chat messages, spreadsheets and memory. The consequences are predictable: unclear priorities, bugs that are reported but not owned, managers without a reliable view of progress, and no record of why something changed. ResolveHQ centralizes this work and keeps an auditable history.

1.2 Target users and personas

Persona	Primary need	Typical role in ResolveHQ
Team owner or agency founder	Visibility across projects, control over who can do what	Owner
Project manager or tech lead	Plan work, assign tasks, track bugs, see workload	Admin
Developer or QA engineer	Know what to do next, report and fix bugs, discuss work	Member
Client or stakeholder	Read-only progress visibility	Viewer

1.3 Domain glossary

Term	Meaning in ResolveHQ
Organization	The tenant. A team, agency or company. All data belongs to exactly one organization (also called a team in the UI).
Membership	Links a user to an organization with one role: owner, admin, member or viewer.
Project	A body of work inside an organization, with a short key (for example ECOM) and its own members.
Project membership	Links a user to a project. Owners and admins implicitly access every project in the organization.
Task	A unit of planned work with assignee, priority, due date and Kanban status.
Issue	A reported defect with severity, priority, reproduction details and the same Kanban status flow.
Activity log	An append-only record of who did what to which resource and when.
Notification	A per-user, in-app message created by an event (assignment, status change, comment).

1.4 Design principles

- **Security is server-enforced.** The frontend hides controls for usability, but only the backend decides what is allowed.
- **Tenant isolation by construction.** Every domain document carries its organization, and every query is scoped by it.
- **Thin HTTP layer, rich services.** Controllers translate HTTP; services hold business rules and are testable without HTTP.
- **Boring, proven technology.** A modular monolith with one database is easier to build, test, deploy and explain than microservices.
- **Honest scope.** A small, correct, tested MVP is preferred to a large, fragile one.

2. Technology Stack

The requested stack is used unchanged. No technical reason was found to replace any component. A small number of **supporting libraries** are added because the requirements (validation, security headers, rate limiting, API docs, integration testing) need them.

2.1 Core stack

Layer	Technology	Responsibility	Notes
Frontend	React + Vite	Single-page application, routing, UI state	Static build, hosted on a CDN
Styling	Tailwind CSS	Utility-first, responsive UI	A small set of shared UI components
Backend	Node.js + Express	REST API, middleware, business logic	Assumes Express 5 (async errors are forwarded to the error handler). On Express 4, wrap handlers in an asyncHandler helper.
Database	MongoDB + Mongoose	Persistence, schemas, indexes	Hosted on MongoDB Atlas
Auth	JWT + bcrypt	Stateless access tokens, password hashing	Short-lived access token plus rotating refresh token (Section 6)
API style	REST (JSON)	Resource-oriented endpoints under /api/v1	Versioned prefix, consistent envelopes
Testing	Vitest + Supertest	Unit and HTTP integration tests	Runs against an in-memory MongoDB
API docs	Swagger / OpenAPI	Interactive, versioned API reference	Served at /api/docs in non-production
Version control	Git + GitHub	History, review, CI	GitHub Actions for tests

2.2 Supporting libraries (additions)

Library	Used for	Why it is needed
zod	Request and environment validation	One schema language for body, params, query and config; rejects wrong types (a key NoSQL-injection defense)
helmet	Secure HTTP headers	Cheap baseline hardening
cors	Origin allowlist	Only the known frontend origin may call the API from a browser
express-rate-limit	Rate limiting	Brute-force and abuse protection
cookie-parser	Reading the refresh cookie	Refresh token lives in an httpOnly cookie
pino + pino-http	Structured logging	Fast JSON logs with redaction of secrets
swagger-ui-express + YAML spec	API documentation	Interactive docs generated from an OpenAPI file
mongodb-memory-server	Test database	Real MongoDB behavior in tests without external services
TanStack Query	Frontend server state	Caching, loading and error states, optimistic updates
React Router	Frontend routing	Protected routes, nested layouts
dnd-kit	Drag and drop	Accessible Kanban interactions

2.3 Why a modular monolith

One deployable backend is divided into modules (auth, organizations, projects, tasks, issues, comments, notifications, activity, analytics). Modules may call each other only through their service functions and never through each other's models. This gives most of the organizational benefit of microservices (clear ownership, replaceable parts) without network calls, distributed transactions or operational overhead. A module can be extracted into a service later if a real scaling need appears.

3. Core Product Features

Each feature below states its purpose, required behavior and implementation requirements. **MVP** describes the minimum that must ship; **Later** lists optional enhancements that must not block the MVP.

3.1 Registration, login, logout and account management

Purpose: Let people securely create an account and manage it.

- **MVP:** register (name, email, password; optionally creates a first organization and makes the user its Owner), login, logout, logout from all devices, view and update profile, change password, delete account.
- Email is unique and stored lowercase. Passwords are hashed with bcrypt and never returned by any API.
- Changing a password invalidates all other sessions. A user who is the sole Owner of an organization cannot delete the account until ownership is resolved.
- **Later:** email verification, password reset by email, two-factor authentication, social login.

3.2 Project creation, editing, archiving and deletion

Purpose: Organize work into projects with a lifecycle.

- **MVP:** create (name, unique project key, description), edit, archive and unarchive, soft delete.
- Archived projects are read-only and hidden from default lists. Deleted projects set deletedAt and disappear from every query; they are not physically purged in the MVP.
- Creating a project adds the creator as a project member. Owners and admins access all projects of their organization.
- **Later:** project templates, custom statuses, permanent purge job.

3.3 Team membership and project invitations

Purpose: Bring people into an organization and into specific projects.

- **MVP:** invite by email with a role and optional project list; the system creates a single-use, expiring invitation and returns an acceptance link that the inviter can copy. The invited person registers or logs in with the same email and accepts.
- Admins and Owners add or remove project members, change roles (within limits in Section 6) and revoke pending invitations.
- **Later:** sending the invitation by email through a provider, invitation reminders, domain-based auto-join.

3.4 Task creation, assignment, deadlines, priorities and status

Purpose: Represent planned work and who owns it.

- **MVP:** title, description, assignee (must be a project member), due date, priority (low, medium, high, urgent), labels (max 10), status, position within a column. Each task receives a per-project number (ECOM-12).
- List with filters (status, assignee, priority, due range, text), sorting and pagination.
- **Later:** subtasks, estimates, sprints, recurring tasks, attachments.

3.5 Bug and issue reporting

Purpose: Capture defects with enough information to fix them.

- **MVP:** title, description, steps to reproduce, expected and actual behavior, severity (minor, major, critical, blocker), priority, assignee, reporter, status.
- Issues and tasks are separate collections because their fields and future workflows differ; both share the same status enum so one board can show both.
- **Later:** duplicate detection, environment fields, linking issues to tasks.

3.6 Kanban workflow

Purpose: Visualize and change work status quickly.

- **MVP:** five fixed columns: Backlog, Todo, In Progress, In Review, Done. Cards move between and within columns.

- Order inside a column uses a numeric **position** (midpoint insertion between neighbors, periodic re-balance when gaps become too small).
- The board endpoint returns tasks and issues grouped by status, limited per column with load-more.
- **Later:** custom columns, WIP limits, swimlanes.

3.7 Comments

Purpose: Keep discussion next to the work.

- **MVP:** add, list, edit (author only) and delete (author, or Owner and Admin) comments on tasks and issues. Plain text up to 5,000 characters, rendered escaped.
- **Later:** Markdown, @mentions, reactions, attachments.

3.8 Notifications and unread tracking

Purpose: Tell people what needs their attention.

- **MVP:** in-app notifications for assignment, status change on items you own or created, and new comments on those items. Unread count, mark one or all as read. The client polls the unread count every 30 to 60 seconds.
- Actors never notify themselves.
- **Later:** email digests, user preferences, real-time delivery with Server-Sent Events or WebSockets.

3.9 Dashboard analytics

Purpose: Give managers a quick view of health and workload.

- **MVP:** per project: completion rate, counts by status, overdue items, open work per assignee. Per organization: progress of each accessible project.
- Computed with MongoDB aggregation pipelines over indexed fields; no separate analytics store.
- **Later:** burndown charts, cycle time, exports.

3.10 Activity logs

Purpose: Provide accountability and history.

- **MVP:** append-only records of important actions with actor, action, resource, timestamp and a small before/after diff of whitelisted fields. Project timeline and organization-wide view for Owners and Admins.
- **Later:** export, retention policies, tamper-evident hashing.

3.11 MVP scope and optional enhancements

Area	Essential MVP	Optional future enhancement
Identity	Email and password, refresh rotation, change password	Email verification, password reset, 2FA, SSO
Tenancy	One organization per project set, four roles, invitation links	Ownership transfer, organization deletion, custom roles
Work items	Tasks, issues, fixed Kanban, comments	Subtasks, sprints, custom workflows, attachments
Awareness	In-app notifications (polling), activity log	Email notifications, real-time push, mentions
Insight	Counts, overdue, workload	Burndown, velocity, exports
Operations	Rate limiting, logs, health check, CI tests	Metrics dashboards, audit export, billing

Recommended cut line for a solo developer. If time runs short, finish in this order and stop where quality would suffer: (1) auth, organizations, RBAC; (2) projects, tasks, issues; (3) Kanban, comments, activity log; (4) notifications; (5) analytics. A complete and tested items 1 to 3 is a stronger deliverable than ten half-working features.

4. System Architecture and Request Handling

4.1 High-level architecture

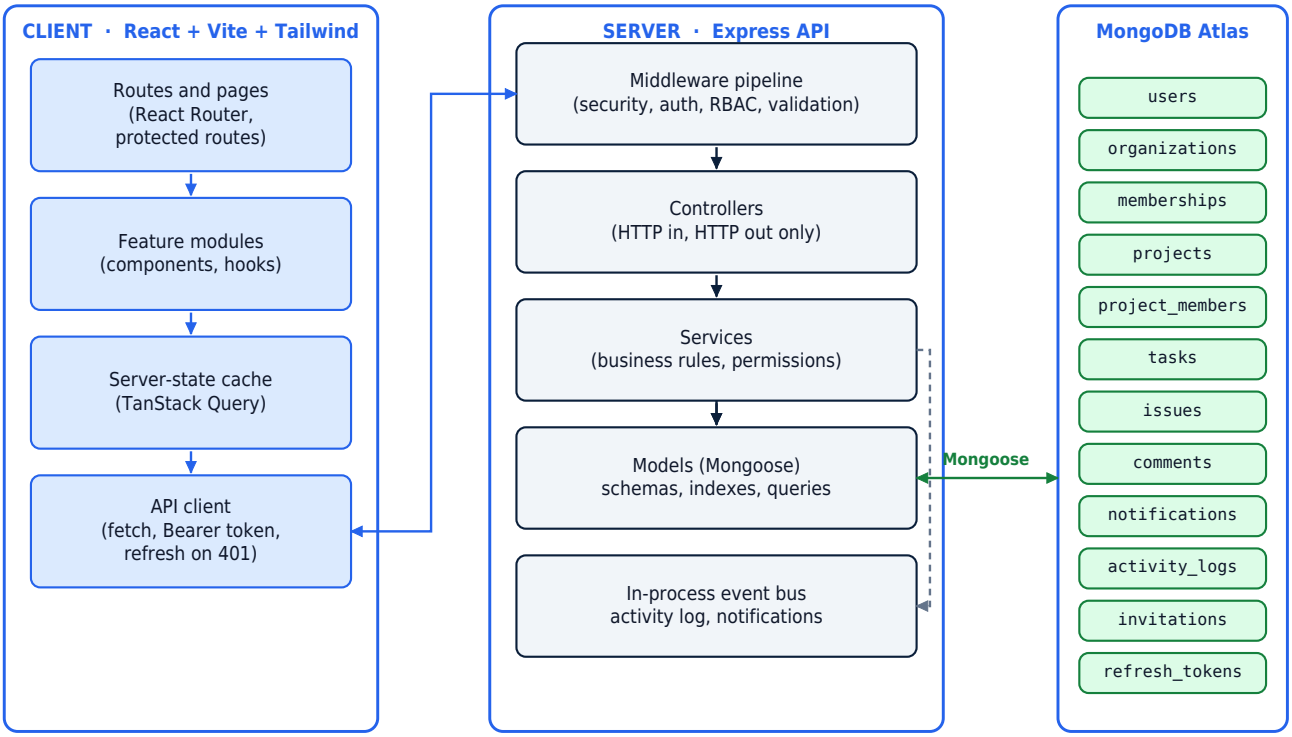


Figure 1. High-level architecture. Browser to API: HTTPS, JSON, REST. API to database: Mongoose driver.

The browser loads a static React build. All business data flows through the REST API; the browser never talks to MongoDB. Inside the server, a request passes through middleware, a controller, a service and finally the model layer. Services emit in-process domain events; the activity and notification modules subscribe to those events.

4.2 Layer responsibilities

Layer	Does	Must not
Routes	Map URL and method to middleware chain and controller	Contain logic
Middleware	Cross-cutting concerns: security headers, auth, context loading, authorization, validation, errors	Query domain data beyond context loading
Controller	Read validated input from req, call one service function, choose the HTTP status, shape the response	Contain business rules or touch models directly
Service	Business rules, orchestration, calls to models, emitting events; receives plain arguments and a context object	Import Express types or read req/res
Model	Mongoose schema, indexes, small query helpers	Know about HTTP or the current user
Event subscribers	Create activity records and notifications from events	Change the primary resource

4.3 Backend project structure

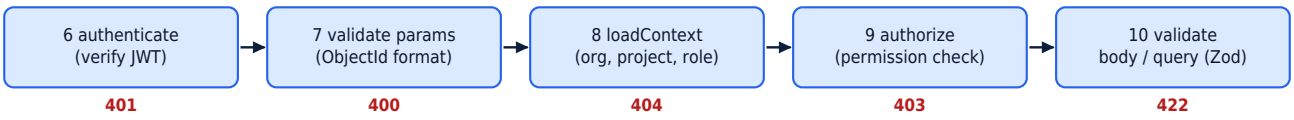
```
server/
  src/
    app.js                # builds the Express app (no listen) - imported by tests
    server.js             # connects DB, starts listening
    config/               # env.js (Zod-validated), db.js, logger.js
    middleware/           # requestId, authenticate, validate, loadContext, authorize,
                          # rateLimit, notFound, errorHandler
    modules/
      auth/               # routes, controller, service, schema (Zod), model (User, RefreshToken)
      organizations/      # Organization, Membership, Invitation
      projects/           # Project, ProjectMembership
      tasks/              # Task
      issues/             # Issue
      comments/           # Comment
      notifications/      # Notification
      activity/           # ActivityLog + event subscribers
      analytics/          # aggregation queries (read-only)
    events/bus.js         # EventEmitter wrapper with typed event names
    utils/                # ApiError classes, pagination, permissions map
  tests/                  # unit/, integration/, helpers/ (factories, auth helpers)
  openapi.yaml
client/
  src/
    app/                  # router, providers, protected route wrapper
    features/<module>/    # api.js, hooks.js, components/, pages/
    lib/apiClient.js      # fetch wrapper
    components/ui/        # shared buttons, modal, table, toast
```

4.4 Express middleware execution order

A. Global middleware (every request)



B. Route-level middleware (protected routes)



C. Application layers and error handling

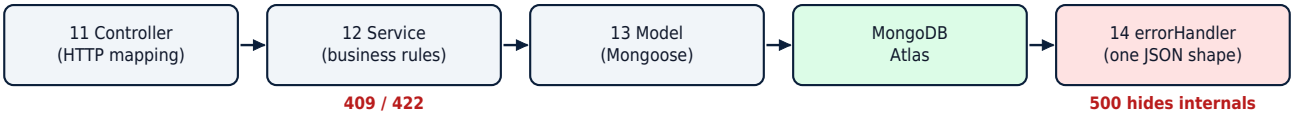


Figure 2. Middleware order. Red labels show the status code returned when a step rejects the request.

#	Middleware	Purpose and key detail
1	requestId + pino-http	Assigns a UUID returned as X-Request-Id and included in error bodies; logs method, path, status, duration. Authorization headers and cookies are redacted.
2-3	helmet, cors	Secure headers; CORS allows only CLIENT_ORIGIN with credentials.
4	rate limiter	Global limit plus a much stricter limit on login, register and refresh.
5	express.json + cookie-parser	JSON body capped at 10 kb to limit abuse.
6	authenticate	Verifies the Bearer JWT, loads the user, rejects if the token version no longer matches.
7	validate params	Checks ObjectId format first so malformed ids return 400 instead of a database cast error.
8	loadContext	Resolves the resource (for example the task), derives its project and organization from stored data (never from client input), verifies organization and project membership and attaches req.ctx = { user, org, project, role }. Returns 404 when the resource is not visible to the user.

#	Middleware	Purpose and key detail
9	authorize(permission)	Checks the role against the permission matrix (Section 6.6); 403 on failure.
10	validate body / query	Zod schema, strict mode; strips unknown keys; 422 with field errors. Runs after authorization so unauthorized callers learn nothing about the schema.
11-13	controller, service, model	See 4.2.
14	errorHandler	Maps known errors to status codes, logs unknown errors, returns the standard error shape without stack traces in production.

4.5 Browser request lifecycle: moving a Kanban card

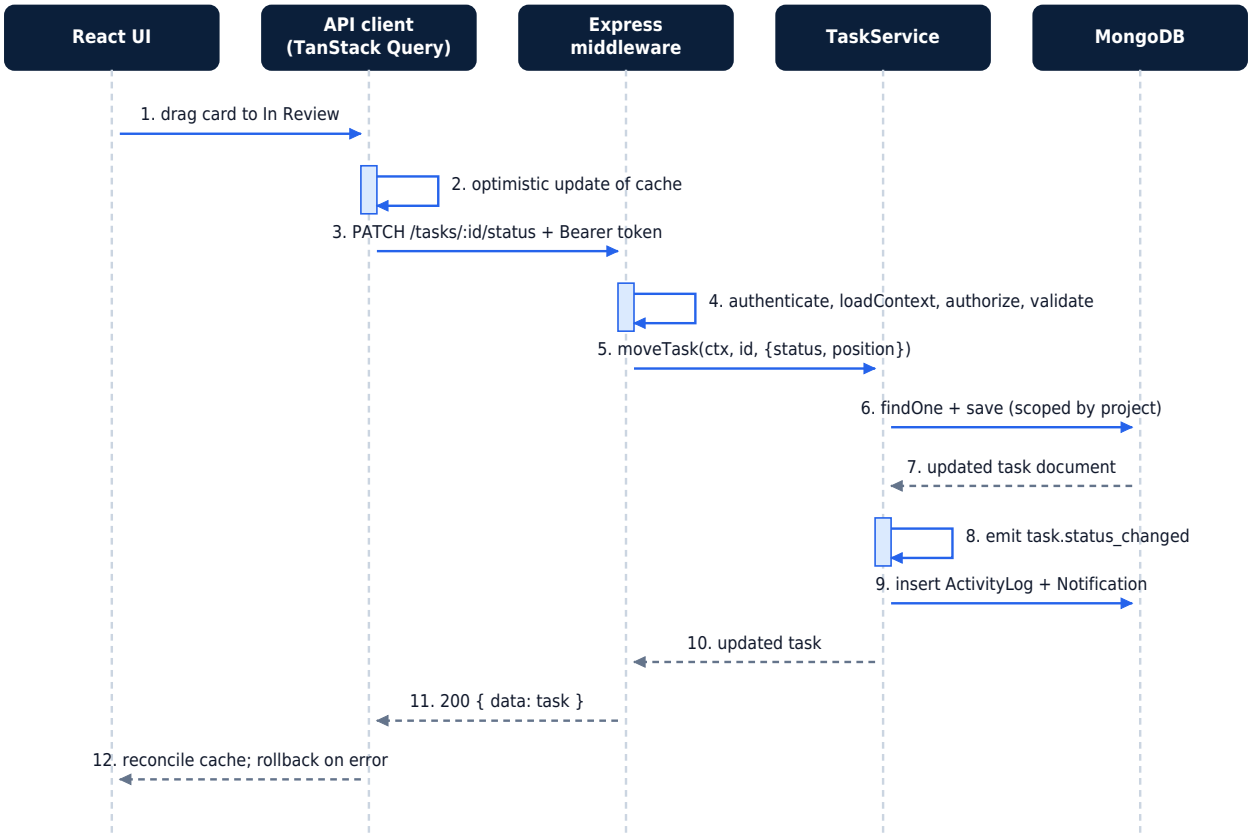


Figure 3. Complete flow of PATCH /api/v1/tasks/:id/status.

This flow shows the pattern used by every mutating endpoint. The client updates its cache optimistically so the card moves instantly. The server independently re-verifies identity, tenant membership, permission and input, performs the change in the service, publishes a domain event, and responds. If any step fails, the client restores the previous cache state and shows the error message from the standard error response.

4.6 Separating business logic from HTTP handling

```
// modules/tasks/task.routes.js
router.patch('/tasks/:taskId/status',
  authenticate, validateParams(idParams('taskId')),
  loadContext({ from: 'task' }), authorize('task:move'),
  validate(moveTaskSchema), taskController.moveTask);
// modules/tasks/task.controller.js  - HTTP only
export const moveTask = async (req, res) => {
  const task = await taskService.moveTask(req.ctx, req.params.taskId, req.body);
  res.status(200).json({ data: task });
};
// modules/tasks/task.service.js  - no req / res
export async function moveTask(ctx, taskId, { status, position }) {
  const task = await Task.findOne({ _id: taskId, project: ctx.project._id });
  if (!task) throw new NotFoundError('Task');
  const before = { status: task.status, position: task.position };
  task.status = status;
  if (position !== undefined) task.position = position;
  await task.save();
  bus.emit('task.status_changed', { ctx, task, before });
  return task;
}
```

Because the service receives a plain context object, it can be unit-tested by calling it directly, and the same function could later be reused by a background job or a WebSocket handler.

4.7 Frontend request handling

- **One API client.** All calls go through lib/apiClient.js, which adds the Authorization header, sends cookies, parses the standard error shape into an ApiError, and performs a single-flight token refresh when it receives 401.
- **Server state in TanStack Query.** Each feature exposes hooks such as useTasks(projectId) that return data, isLoading and error. Mutations invalidate or optimistically update the relevant query keys.
- **Loading and errors.** Skeleton placeholders while loading, inline messages for 422 field errors, toasts for 403 and 5xx, and a redirect to login when refresh fails.
- **Protected routes.** A ProtectedRoute wrapper redirects unauthenticated users. It is a usability feature only; the API remains the authority.
- **Same-origin API calls.** The frontend calls relative /api/v1 URLs. In production the host proxies /api to the backend (Section 12), avoiding cross-site cookie problems.

```
// client/src/lib/apiClient.js  (illustrative)
let accessToken = null;                          // memory only - never localStorage
export const setAccessToken = (t) => { accessToken = t; };
export async function api(path, { method = 'GET', body, retry = true } = {}) {
  const res = await fetch(`/api/v1${path}`, {
    method, credentials: 'include',
    headers: { 'Content-Type': 'application/json',
               ...(accessToken && { Authorization: `Bearer ${accessToken}` }) },
    body: body ? JSON.stringify(body) : undefined,
  });
  if (res.status === 401 && retry && path !== '/auth/refresh') {
    if (await refreshAccessToken()) return api(path, { method, body, retry: false });
  }
  const json = res.status === 204 ? null : await res.json();
  if (!res.ok) throw new ApiError(res.status, json.error);
  return json;
}
```

4.8 Environment variables and configuration

Variable	Side	Example / rule
NODE_ENV	server	development | test | production
PORT	server	4000
MONGODB_URI	server	Atlas connection string. Secret. Never committed.
JWT_ACCESS_SECRET	server	At least 32 random bytes. Secret. Different per environment.
ACCESS_TOKEN_TTL	server	15m
REFRESH_TOKEN_TTL_DAYS	server	7
BCRYPT_COST	server	12 (tune so hashing takes roughly 250 ms on the host)
CLIENT_ORIGIN	server	https://app.example.com - used for CORS and invitation links
LOG_LEVEL	server	info
VITE_APP_NAME and other VITE_* variables	client	Public by definition: every VITE_* value is visible in the browser bundle. Never put secrets here.

```
// server/src/config/env.js - the app refuses to start if configuration is wrong
import { z } from 'zod';
const schema = z.object({
  NODE_ENV: z.enum(['development', 'test', 'production']).default('development'),
  PORT: z.coerce.number().default(4000),
  MONGODB_URI: z.string().min(1),
  JWT_ACCESS_SECRET: z.string().min(32),
  ACCESS_TOKEN_TTL: z.string().default('15m'),
  REFRESH_TOKEN_TTL_DAYS: z.coerce.number().default(7),
  BCRYPT_COST: z.coerce.number().min(10).max(14).default(12),
  CLIENT_ORIGIN: z.string().url(),
});
export const env = schema.parse(process.env);
```

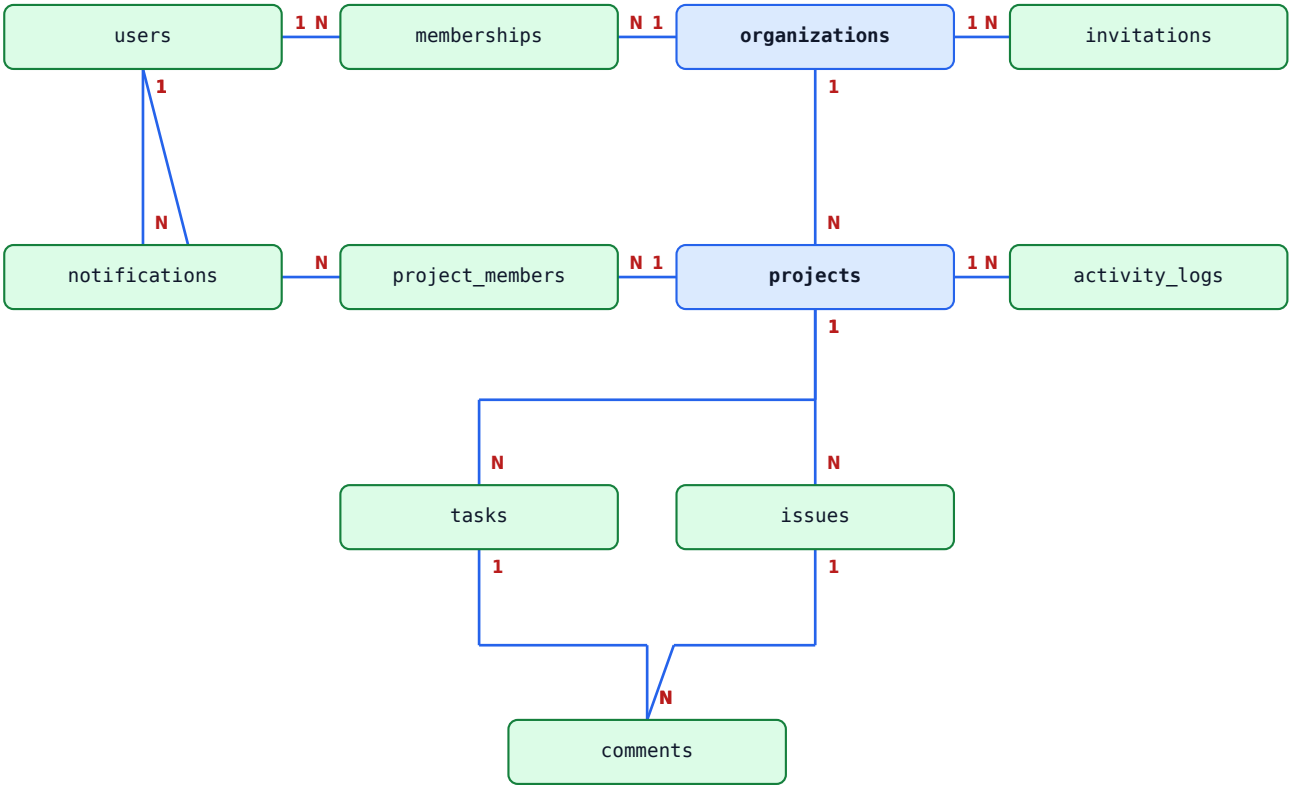
A committed **.env.example** lists every variable with a placeholder. The real **.env** is listed in .gitignore. Production secrets are set in the hosting provider's dashboard.

5. Database and Data Relationships

ResolveHQ uses MongoDB through Mongoose. MongoDB is a **document database**, not a relational one: it does not enforce foreign keys, joins are optional and explicit, and relationships are modeled by choosing between references and embedded data. This section defines every collection and explains each modeling choice.

> **Two collections beyond the requested list.** *invitations* and *refresh_tokens* are added because the invitation flow and secure session rotation both need persistent state. All other collections match the requested data model (Organizations are the teams; the join collections are *memberships* and *project_members*).

5.1 Entity relationship diagram



Legend: blue = tenant and project roots, green = other collections. 1 / N = one-to-many.

Figure 4. Collection relationships. References to users (assignee, author, actor, creator) are omitted for clarity.

5.2 Collection definitions

Conventions: every collection has an automatic **_id** (ObjectId). Collections marked [T] also have Mongoose **timestamps** (createdAt, updatedAt). Domain collections carry a denormalized **organization** reference for tenant isolation. "Req" means required.

users [T]

Field	Type	Req	Rules and notes
name	String	Y	Trimmed, 1 to 80 characters.
email	String	Y	Lowercased, trimmed, valid format. Unique.
passwordHash	String	Y	bcrypt hash. select:false so it is never loaded by default.
tokenVersion	Number	Y	Default 0. Incremented to revoke every access token of the user.
status	String	Y	active | deleted. Deleted accounts are anonymized (Section 5.5).
lastLoginAt	Date	N	Updated on successful login.

Indexes: `{ email: 1 }` unique.

organizations [T]

Field	Type	Req	Rules and notes
name	String	Y	1 to 80 characters.
slug	String	Y	URL-safe, unique.
createdBy	ObjectId	Y	Reference to users.

Indexes: `{ slug: 1 }` unique.

memberships [T] (organization membership, many-to-many join)

Field	Type	Req	Rules and notes
organization	ObjectId	Y	Reference to organizations.
user	ObjectId	Y	Reference to users.
role	String	Y	owner | admin | member | viewer.
invitedBy	ObjectId	N	Reference to users.

Indexes: `{ organization: 1, user: 1 }` unique; `{ user: 1 }`; `{ organization: 1, role: 1 }`. Invariant: every organization has at least one owner.

projects [T]

Field	Type	Req	Rules and notes
organization	ObjectId	Y	Reference to organizations.
name	String	Y	1 to 100 characters.
key	String	Y	2 to 6 uppercase letters or digits (ECOM). Immutable after creation.
description	String	N	Up to 2,000 characters.
status	String	Y	active | archived.
counter	Number	Y	Default 0. Atomically incremented with $inc to assign the next task or issue number.
createdBy	ObjectId	Y	Reference to users.
deletedAt	Date	N	Soft delete marker. Null when live.

Indexes: `{ organization: 1, key: 1 }` unique (includes deleted projects); `{ organization: 1, deletedAt: 1, status: 1, createdAt: -1 }`.

project_members [T] (project membership, many-to-many join)

Field	Type	Req	Rules and notes
organization	ObjectId	Y	Denormalized for tenant scoping.
project	ObjectId	Y	Reference to projects.
user	ObjectId	Y	Reference to users. Must be a member of the same organization.
addedBy	ObjectId	Y	Reference to users.

Indexes: `{ project: 1, user: 1 }` unique; `{ user: 1, organization: 1 }` (list my projects).

tasks [T]

Field	Type	Req	Rules and notes
organization, project	ObjectId	Y	References. project must belong to organization.
number	Number	Y	From projects.counter. Display key is project.key + '-' + number.
title	String	Y	1 to 200 characters.
description	String	N	Up to 10,000 characters.
status	String	Y	backlog | todo | in_progress | in_review | done. Default backlog.
position	Number	Y	Order inside a column (ascending).
priority	String	Y	low | medium | high | urgent. Default medium.
assignee	ObjectId	N	Reference to users; must be a project member.
createdBy	ObjectId	Y	Reference to users.
dueDate	Date	N	Stored in UTC.
labels	[String]	N	Embedded. Max 10 items of up to 30 characters.
completedAt	Date	N	Set when status becomes done, cleared if it leaves done.

Indexes: `{ project: 1, number: 1 }` unique; `{ project: 1, status: 1, position: 1 }` (board); `{ project: 1, assignee: 1, status: 1 }` (workload); `{ project: 1, dueDate: 1 }` (overdue); `{ assignee: 1, status: 1, dueDate: 1 }` (my work).

issues [T] (same fields as tasks plus the following)

Field	Type	Req	Rules and notes
severity	String	Y	minor | major | critical | blocker.
stepsToReproduce	String	N	Up to 5,000 characters.
expectedBehavior, actualBehavior	String	N	Up to 2,000 characters each.
createdBy	ObjectId	Y	Acts as the reporter.

Indexes: Same as tasks, plus `{ project: 1, severity: 1, status: 1 }`. Numbers share the project counter, so keys never collide with tasks.

comments [T]

Field	Type	Req	Rules and notes
organization, project	ObjectId	Y	Denormalized for scoping.
targetType	String	Y	task | issue.
target	ObjectId	Y	Id of the task or issue (Mongoose refPath on targetType).
author	ObjectId	Y	Reference to users.
body	String	Y	1 to 5,000 characters, stored raw, rendered escaped.
editedAt	Date	N	Set on edit.

Indexes: `{ targetType: 1, target: 1, createdAt: 1 }` (thread); `{ author: 1 }`.

notifications (createdAt only)

Field	Type	Req	Rules and notes
recipient	ObjectId	Y	Reference to users.
organization, project	ObjectId	Y	For filtering and deep links.
type	String	Y	task_assigned | issue_assigned | status_changed | comment_added | removed_from_project | invitation_accepted.
actor	ObjectId	Y	User who caused it.
targetType, target	String, ObjectId	N	Resource to open when clicked.
message	String	Y	Short pre-rendered text, max 200 characters.
readAt	Date	N	Null means unread.

Indexes: `{ recipient: 1, readAt: 1, createdAt: -1 }` (unread list and count); TTL index on createdAt (for example 180 days).

activity_logs (createdAt only, append-only)

Field	Type	Req	Rules and notes
organization	ObjectId	Y	Tenant.
project	ObjectId	N	Null for organization-level events.
actor	ObjectId	Y	User who performed the action.
action	String	Y	Dotted name such as task.status_changed.
resourceType, resourceId	String, ObjectId	Y	Affected resource.
summary	String	Y	Human-readable snapshot (for example the task title) so history survives deletions.
changes	{ before, after }	N	Embedded, whitelisted fields only (status, assignee, priority, title, dueDate, role).
requestId	String	N	Correlates with application logs.

Indexes: `{ project: 1, createdAt: -1 }`; `{ organization: 1, createdAt: -1 }`; `{ resourceType: 1, resourceId: 1, createdAt: -1 }`.

invitations [T]

Field	Type	Req	Rules and notes
organization	ObjectId	Y	Tenant.
email	String	Y	Lowercased. The accepting user must log in with this email.
role	String	Y	admin | member | viewer. Never owner.
projects	[ObjectId]	N	Projects to join on acceptance (max 20).
tokenHash	String	Y	SHA-256 of the random token. The raw token exists only in the link.
status	String	Y	pending | accepted | revoked | expired.
expiresAt	Date	Y	Seven days after creation.
invitedBy, acceptedBy	ObjectId	N	References to users.

Indexes: `{ tokenHash: 1 }` unique; `{ organization: 1, email: 1, status: 1 }`; TTL on expiresAt (cleanup after 30 days).

refresh_tokens (createdAt only)

Field	Type	Req	Rules and notes
user	ObjectId	Y	Reference to users.
tokenHash	String	Y	SHA-256 of the refresh token. The raw token is only in the cookie.
family	String	Y	Random id shared by a rotation chain; used for reuse detection.
expiresAt	Date	Y	Seven days after issue.
revokedAt	Date	N	Set on rotation, logout or revocation.

Indexes: `{ tokenHash: 1 }` unique; `{ user: 1 }`; TTL on expiresAt.

5.3 Relationships and how MongoDB represents them

Relationship	Cardinality	Representation
Organization to projects	One-to-many	projects.organization holds the owning organization's ObjectId (reference).
Users to organizations	Many-to-many	Join collection memberships with the extra attribute role.
Users to projects	Many-to-many	Join collection project_members.
Project to tasks and issues	One-to-many	tasks.project and issues.project (reference).
Task or issue to comments	One-to-many, polymorphic	comments.targetType plus comments.target.
Task or issue to assignee	Many-to-one	assignee ObjectId on the task or issue.
User to notifications	One-to-many	notifications.recipient (reference).
Task to labels	One-to-few	Embedded string array.
Activity entry to changes	One-to-one	Embedded before/after object.

References versus embedding

- **Embed** when data is small, bounded, owned by the parent and always read with it: labels, the before/after diff.
- **Reference** when data is unbounded, queried or paginated on its own, or shared: comments, tasks, activity, members. A members array inside a project would be unbounded, impossible to index for "my projects", and would make concurrent updates conflict.
- **Denormalize deliberately** only where it buys isolation or speed: the organization id on every domain document, and the summary text in activity logs.
- MongoDB documents are limited to 16 MB, which is another reason never to embed growing lists.

5.4 Referential integrity

MongoDB does not enforce foreign keys, so the service layer is responsible for integrity:

- Before saving a reference (assignee, project member, invitation project list) the service verifies the target exists **and belongs to the same organization**.
- Operations that touch several documents (register: user, organization, membership; remove member: membership, project memberships, unassign work) run inside a **multi-document transaction**. Transactions require a replica set; Atlas clusters provide one.
- Unique compound indexes enforce "one membership per user per organization" and similar rules at the database level, so races cannot create duplicates.
- A small integrity script (run in CI against test data and manually in production) reports orphaned references.

5.5 Deletion strategies

Entity	Strategy	Reason
Project	Soft delete (deletedAt). Every project lookup excludes deleted projects, so children become unreachable.	Recoverable, avoids cascading deletes of thousands of documents in one request.
Task or issue	Hard delete, together with its comments, in one transaction. A summary remains in activity_logs.	Simple and predictable. History is preserved separately.
Comment	Hard delete.	Authorship is recorded in the activity log.
Organization member removed	Delete membership and project memberships in that organization; set assignee to null on their open work; emit events.	Work is not left owned by someone without access.
User account	Anonymize: name becomes "Deleted user", email replaced with a non-routable placeholder, password hash replaced, all tokens revoked, memberships removed. Documents referencing the id are kept.	Preserves history without keeping personal data.
Invitation	Status change; TTL cleanup of old records.	Keeps a short audit trail.
Notification	TTL expiry.	Ephemeral by nature.
Activity log	Never deleted or edited by the application.	Accountability.

5.6 Data ownership and tenant isolation

- The organization is the tenant boundary. Every domain document stores **organization**; every query includes it.
- The tenant is never taken from the request body or URL alone. loadContext derives it from the authenticated user's membership and from the stored resource.
- A query helper makes the safe path the easy path:

```
// utils/scope.js
export const scope = (ctx) => ({ organization: ctx.org._id, project: ctx.project._id });
// usage - cannot forget the tenant filter by accident
Task.find({ ...scope(ctx), status: 'todo' }).sort({ position: 1 }).lean();
```

- Mongoose is configured with **sanitizeFilter: true** so that user-supplied objects containing $ operators are neutralized.
- Automated tests exercise every endpoint with a user from a different organization and expect 404 (Section 10).

5.7 Query performance and indexing

- Indexes follow the actual queries: the board uses (project, status, position); overdue work uses (project, dueDate); workload uses (project, assignee, status).
- Compound index field order puts equality fields first, then sort fields. Each index costs write time and memory, so only indexes tied to a documented query are created.
- Read endpoints use **.lean()** and field projection. Lists are paginated with a maximum page size of 100.
- Activity and notification feeds use **cursor pagination** on (createdAt, _id) so deep pages stay fast; small lists use page and limit.
- Analytics use aggregation pipelines with an initial $match on project (hitting an index) before $group.
- During development, run explain('executionStats') on the board, overdue and activity queries and confirm they use IXSCAN rather than COLLSCAN.

6. Authentication and Authorization

6.1 Design summary

Concern	Decision
Password storage	bcrypt, cost 12, per-hash salt (built in).
Access token	Signed JWT (HS256), 15 minute lifetime, held in browser memory.
Session continuity	Refresh token: random 256-bit value in an httpOnly, Secure, SameSite cookie scoped to /api/v1/auth. Only its SHA-256 hash is stored. Rotated on every use.
Invalidation	Logout revokes the refresh token. Logout-all, password change and account deletion increment users.tokenVersion and revoke all refresh tokens.
Authorization	Role-based, enforced in middleware and services, using organization role plus project membership.

6.2 Registration, login, refresh and logout flow

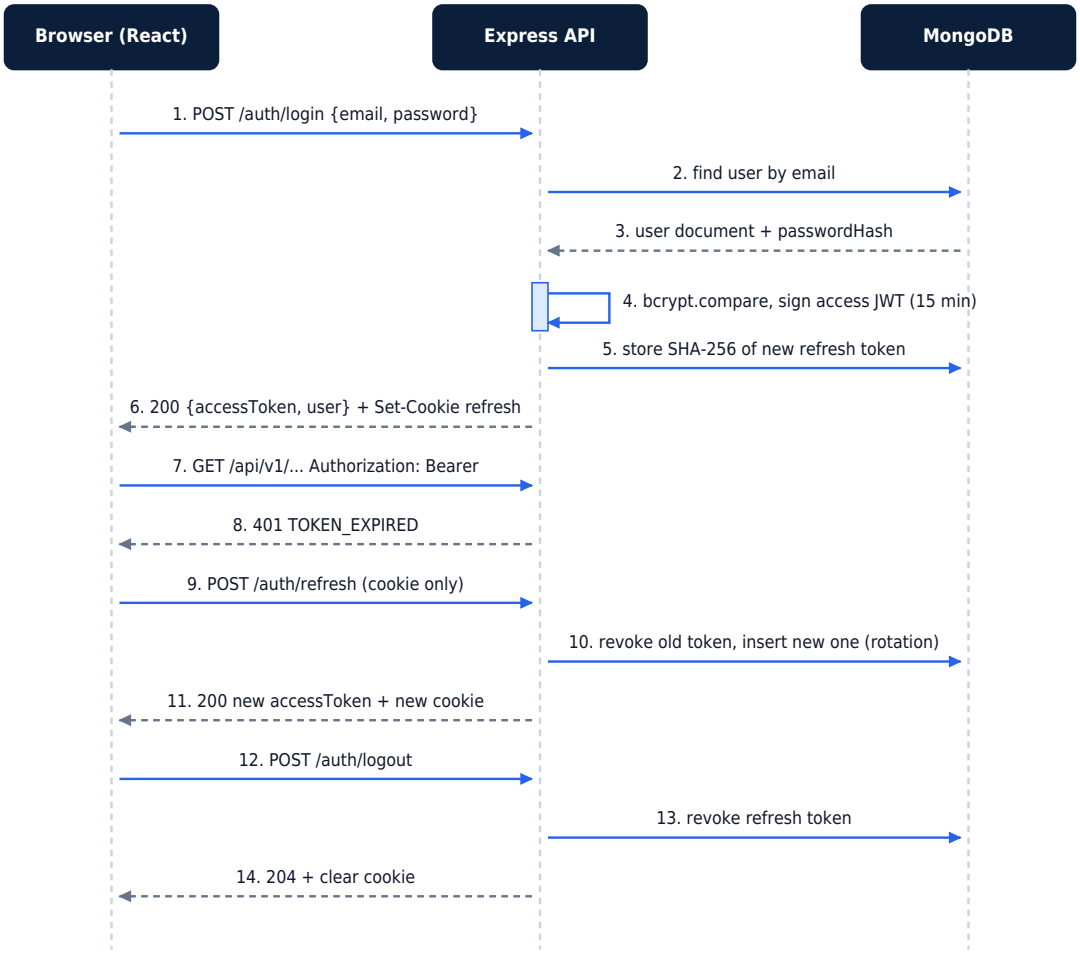


Figure 5. Authentication and session lifecycle.

6.3 Registration and password handling

- Validate: email format, name length, **password length 10 to 72 bytes**. bcrypt only uses the first 72 bytes, so longer inputs must be rejected rather than silently truncated. Favor length over arbitrary complexity rules.
- Normalize email to lowercase before lookup and storage.
- Hash with bcrypt (cost from BCRYPT_COST). Never log or return the password or the hash.
- Registration creates user, organization and an owner membership in one transaction.
- **Login uses one generic error** ("Invalid email or password"). When the email is unknown, the server still runs bcrypt.compare against a fixed dummy hash so response time does not reveal which emails exist.

- Registering an already used email returns 409. This reveals that the email exists; the MVP accepts that trade-off and relies on rate limiting. Email verification (later) removes the issue.

6.4 JWT creation, verification and expiration

```
// access token payload - deliberately minimal
{ "sub": "665f...a1", "ver": 3, "iat": 1767450000, "exp": 1767450900, "iss": "resolvehq-api", "aud":
    "resolvehq-web" }
jwt.sign(payload, env.JWT_ACCESS_SECRET, { algorithm: 'HS256', expiresIn: env.ACCESS_TOKEN_TTL });
jwt.verify(token, env.JWT_ACCESS_SECRET, { algorithms: ['HS256'], issuer: 'resolvehq-api', audience:
    'resolvehq-web' });
```

- **No role or organization in the token.** Roles change; they are loaded from the database on each request so a removed member loses access immediately.
- Always pass the allowed algorithms to verify so tokens signed with "none" or another algorithm are rejected.
- Expired tokens return 401 with code TOKEN_EXPIRED so the client knows to refresh; invalid tokens return 401 INVALID_TOKEN.

6.5 Token storage and handling

Token	Where it lives	Why
Access token	JavaScript memory in the SPA only	Not readable by other tabs or persisted storage. Not exposed to theft via localStorage-reading scripts. Lost on reload, which is fine because of silent refresh.
Refresh token	httpOnly, Secure, SameSite=Strict cookie, Path=/api/v1/auth	JavaScript cannot read it, so an XSS bug cannot steal it. Sent only to the auth endpoints.

- On page load the SPA calls POST /auth/refresh. If it succeeds, the user is logged in without typing credentials; then it calls GET /users/me.
- **Rotation and reuse detection:** each refresh revokes the presented token and issues a new one in the same family. If a token that is already revoked is presented again, the whole family is revoked, because it was probably stolen.
- **CSRF:** only /auth/refresh and /auth/logout use the cookie. They are POST-only, the cookie is SameSite=Strict, and the server also checks that the Origin header equals CLIENT_ORIGIN. Every other endpoint uses the Authorization header, which browsers do not attach automatically.
- Cookies require HTTPS in production. Serving the API through the same origin as the frontend (proxy rewrite, Section 12) keeps the cookie first-party.

6.6 Logout and token invalidation

Event	Server action	Effect on existing access tokens
Logout (this device)	Revoke the refresh token, clear the cookie	Still valid until expiry (maximum 15 minutes). The client discards it immediately.
Logout everywhere	tokenVersion += 1 and revoke all refresh tokens of the user	Rejected on the next request (version mismatch).
Password change	Same as logout everywhere, then issue a fresh session for the current device	Rejected on the next request.
Account deletion	Same as logout everywhere, then anonymize	Rejected.
Refresh token reuse detected	Revoke the family and increment tokenVersion	Rejected.

The 15 minute window after a single-device logout is the standard trade-off of stateless access tokens. authenticate loads the user on every request (one indexed read) specifically so that tokenVersion and account status can be checked.

6.7 Backend authentication middleware

```
// middleware/authenticate.js
export async function authenticate(req, res, next) {
  const header = req.get('authorization') || '';
  const [scheme, token] = header.split(' ');
  if (scheme !== 'Bearer' || !token) throw new UnauthenticatedError('MISSING_TOKEN');
  let payload;
  try {
    payload = jwt.verify(token, env.JWT_ACCESS_SECRET, { algorithms: ['HS256'], issuer: 'resolvehq-api',
        audience: 'resolvehq-web' });
  } catch (e) {
    throw new UnauthenticatedError(e.name === 'TokenExpiredError' ? 'TOKEN_EXPIRED' : 'INVALID_TOKEN');
  }
  const user = await User.findById(payload.sub).select('name email tokenVersion status');
  if (!user || user.status !== 'active' || user.tokenVersion !== payload.ver) throw new
      UnauthenticatedError('INVALID_TOKEN');
  req.user = user;
  next();
}
```

6.8 Protected frontend routes

- AuthProvider bootstraps the session (refresh, then users/me) and exposes user, login, logout and the current organization role.
- ProtectedRoute renders a loading state while bootstrapping, redirects to /login when there is no session, and remembers the intended URL.
- A can(permission) helper, fed by the same permission map as the server (a copy generated from one shared source or documented identically), hides or disables controls the user cannot use.
- A 403 from the API is displayed as "You do not have permission" and never treated as a logout. A 401 that survives the refresh attempt logs the user out.

6.9 Account and session security

- Rate limits: for example 10 login attempts per 15 minutes per IP and per email, and tighter limits on register and refresh.
- Changing a password or deleting an account requires the current password.
- Security events (failed login, refresh reuse, role change) are logged with request id but without secrets.
- Cookies: httpOnly, Secure, SameSite=Strict. HSTS via helmet in production.
- Later: account lockout with backoff, session list and device revocation, 2FA.

6.10 Role-based access control (RBAC)

Role	Scope	Description
Owner	Organization	Full control of the organization and every project. At least one Owner always exists. The Owner role cannot be changed or removed in the MVP (no ownership transfer).
Admin	Organization	Administers members and projects. Implicitly accesses every project. Cannot touch Owners or other Admins, cannot delete projects.
Member	Projects they belong to	Creates and works on tasks, issues and comments in joined projects.
Viewer	Projects they belong to	Read-only access to joined projects.

Effective access rule. A user may access a project when (a) they have an organization membership and (b) they are an Owner or Admin, or have a project_members record for that project. Their organization role then determines which actions are allowed.

Every check must pass, in this order. Checks 2, 3 and 5 return 404 so that the existence of other tenants' data is not revealed.

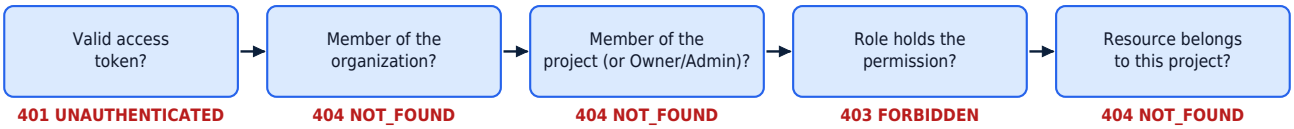


Figure 6. Authorization decision chain applied to every protected request.

6.11 Permission matrix

Yes = allowed. Own = allowed only on items the user created. Limited = allowed with the restriction stated. No = denied. "Joined projects" means projects where the user has a project membership; Owners and Admins are treated as members of all.

Action	Owner	Admin	Member	Viewer
Update organization settings	Yes	Yes	No	No
Invite members, revoke invitations	Yes	Yes (not Admin role)	No	No
Change a member's role	Yes (not own Owner role)	Limited: Member and Viewer only	No	No
Remove a member from the organization	Yes (not self if last Owner)	Limited: Member and Viewer only	No	No
Create a project	Yes	Yes	No	No
Edit or archive a project	Yes	Yes	No	No
Delete a project	Yes	No	No	No
Add or remove project members	Yes	Yes	No	No
View a project, board and lists	Yes (all)	Yes (all)	Yes (joined)	Yes (joined)
Create tasks and issues	Yes	Yes	Yes (joined)	No
Edit tasks and issues, change fields	Yes	Yes	Yes (joined)	No
Move cards, assign work	Yes	Yes	Yes (joined)	No
Delete tasks and issues	Yes	Yes	Own	No
Add comments	Yes	Yes	Yes (joined)	No
Edit a comment	Own	Own	Own	No
Delete a comment	Yes	Yes	Own	No
View project activity log	Yes	Yes	Yes (joined)	Yes (joined)
View organization-wide activity log	Yes	Yes	No	No
View project analytics	Yes	Yes	Yes (joined)	Yes (joined)
Read and clear own notifications	Yes	Yes	Yes	Yes
Manage own account	Yes	Yes	Yes	Yes

6.12 Enforcing authorization in code

```
// utils/permissions.js - single source of truth
export const ROLE_PERMISSIONS = {
  owner:  ['*'],
  admin:  ['org:update', 'member:invite', 'member:role', 'member:remove', 'project:create',
      'project:update',
           'project:archive', 'projectMember:manage', 'project:read', 'task:*', 'issue:*',
               'comment:create',
           'comment:update:own', 'comment:delete', 'activity:project', 'activity:org', 'analytics:read'],
  member: ['project:read', 'task:create', 'task:update', 'task:move', 'task:assign', 'task:delete:own',
           'issue:create', 'issue:update', 'issue:move', 'issue:assign', 'issue:delete:own',
               'comment:create',
           'comment:update:own', 'comment:delete:own', 'activity:project', 'analytics:read'],
  viewer: ['project:read', 'activity:project', 'analytics:read'],
};
export const can = (role, perm) => { const p = ROLE_PERMISSIONS[role] || [];
  return p.includes('*') || p.includes(perm) || p.includes(perm.split(':')[0] + ':*'); };
// middleware/authorize.js
export const authorize = (perm) => (req, res, next) => {
  if (!can(req.ctx.role, perm)) throw new ForbiddenError();
  next();
};
// "own" and role-limit rules live in services, where the resource is loaded
if (!can(ctx.role, 'task:delete') && !(can(ctx.role, 'task:delete:own') &&
    task.createdBy.equals(ctx.user._id)))
  throw new ForbiddenError();
```

Protection against unauthorized cross-project access

- The project is derived from the stored resource and checked against the user's memberships before any handler runs.
- Service queries always include the project and organization from ctx, even when an id is supplied (Section 5.6).
- Assignees, project members and invitation project lists are validated to belong to the same organization (and project, for assignees).
- Resources outside the user's visibility return 404, not 403. Tests assert both that cross-project reads and writes fail.

7. Validation, Security and Error Handling

7.1 Input validation and sanitization

Every request is validated on the server, regardless of any client-side validation. Zod schemas exist for three inputs per route: **params**, **query** and **body**. Schemas are **strict**: unknown keys are rejected or stripped, strings are trimmed, types are enforced (an object where a string is expected is an error), enums are closed and lengths are bounded.

```
// modules/tasks/task.schema.js
export const createTaskSchema = z.object({
  title:       z.string().trim().min(1).max(200),
  description: z.string().max(10000).optional(),
  priority:    z.enum(['low', 'medium', 'high', 'urgent']).default('medium'),
  status:      z.enum(['backlog', 'todo', 'in_progress', 'in_review', 'done']).default('backlog'),
  assignee:    objectId.nullable().optional(),
  dueDate:     z.coerce.date().nullable().optional(),
  labels:      z.array(z.string().trim().min(1).max(30)).max(10).default([]),
}).strict();
```

- **Output encoding over input stripping.** User text (titles, comments) is stored as typed and rendered as text by React, which escapes by default. The frontend must never use dangerouslySetInnerHTML for user content. If Markdown is added later, render it through a sanitizer such as DOMPurify.
- Query parameters are validated like bodies: page and limit are bounded integers; sort fields come from an allow-list; filters accept only enums, ids or dates.
- A shared objectId validator checks 24 hexadecimal characters so malformed ids never reach Mongoose.

7.2 Password and credential protection

- bcrypt hashes only; passwordHash is select:false and removed from every serialized response by a model-level toJSON transform.
- Raw refresh tokens, invitation tokens and passwords are never stored or logged. Tokens are random 256-bit values; only SHA-256 hashes are persisted.
- Invitation tokens are single-use and expire after seven days. Lookup is by hash, so a database leak does not expose usable links.
- Generic login errors and equalized timing (Section 6.3).

7.3 CORS, headers, rate limiting and brute-force protection

Control	Configuration (initial values, adjustable)
CORS	origin: CLIENT_ORIGIN only; credentials: true; methods GET, POST, PUT, PATCH, DELETE; no wildcard origin.
helmet	Default secure headers, HSTS in production, a restrictive Content-Security-Policy for the API's docs page.
Global rate limit	For example 300 requests per 15 minutes per IP, returning 429 with Retry-After.
Auth rate limit	For example 10 login attempts per 15 minutes per IP and per email; 5 registrations per hour per IP.
Body size	express.json limit 10 kb.
Trust proxy	Set trust proxy correctly behind the host's load balancer so rate limiting sees the real client IP.
Brute-force	Rate limit plus bcrypt cost; later: temporary lockout with exponential backoff.

7.4 NoSQL injection and unsafe input

A classic attack sends `{"email": {"$gt": ""}, "password": {"$gt": ""}}` hoping the server passes the body straight into a query. Defenses, in order of importance:

- **Type-strict validation:** Zod requires email and password to be strings, so objects are rejected with 422.
- **sanitizeFilter: true** (Mongoose) wraps $-prefixed keys in $eq as a second layer.

- **Never spread req.body or req.query** into a query or an update. Build queries from named, validated fields only.
- Do not use $where or server-side JavaScript. Escape user text before building a regular expression for search, or use anchored, length-limited patterns.
- Update endpoints use an allow-list of editable fields, which also prevents mass-assignment (for example setting role, organization or createdBy).

7.5 Environment variables and secrets

- Secrets exist only in environment variables, validated at startup. They are never committed, logged or sent to the client.
- Separate secrets and databases for development, test and production. Rotate JWT_ACCESS_SECRET if exposure is suspected (this logs everyone out).
- Enable GitHub secret scanning, and add a pre-commit check for accidental secrets. Keep .env in .gitignore from the first commit.

7.6 Ownership checks and IDOR prevention

Insecure direct object reference (IDOR) means a user changes an id in a URL and reaches someone else's data. Prevention in ResolveHQ:

- Ids in URLs are never trusted for authorization. loadContext resolves the resource and verifies the caller's organization and project membership.
- All service queries include organization and project from ctx (the scope helper).
- Assigning a user, adding a project member or inviting to projects validates that the target belongs to the same organization.
- Rules about "own" resources (delete task, edit comment) compare createdBy or author with the authenticated user in the service.
- Not-visible and non-existent resources both return 404. A dedicated test suite replays every endpoint as a user from another organization and as a user from another project.

7.7 Centralized error handling

```
// utils/errors.js
export class ApiError extends Error {
  constructor(status, code, message, details) { super(message); Object.assign(this, { status, code,
      details }); }
}
export class NotFoundError extends ApiError { constructor(what = 'Resource') { super(404, 'NOT_FOUND',
    `${what} not found`); } }
export class ForbiddenError extends ApiError { constructor() { super(403, 'FORBIDDEN', 'You do not have
    permission to do this'); } }
export class UnauthenticatedError extends ApiError { constructor(code = 'INVALID_TOKEN') { super(401,
    code, 'Authentication required'); } }
// middleware/errorHandler.js
export function errorHandler(err, req, res, next) {
  const known = err instanceof ApiError;
  if (!known) req.log.error({ err }, 'unhandled error');           // full detail only in logs
  const status = known ? err.status : 500;
  res.status(status).json({ error: {
    code: known ? err.code : 'INTERNAL',
    message: known ? err.message : 'Something went wrong',          // no stack, no internals
    details: known ? err.details : undefined,
    requestId: req.id } });
}
```

Status	Code	Used when
200 / 201 / 204	-	Success / created (with Location header) / success with no body.
400	BAD_REQUEST	Malformed JSON or malformed id in the URL.
401	UNAUTHENTICATED, MISSING_TOKEN, INVALID_TOKEN, TOKEN_EXPIRED, INVALID_CREDENTIALS	No, invalid or expired credentials. The client may try to refresh.

Status	Code	Used when
403	FORBIDDEN	Authenticated and in the right tenant, but the role does not allow the action.
404	NOT_FOUND	Resource does not exist or is not visible to the caller.
409	CONFLICT	Duplicate (email, project key, membership), last-owner rule, stale state.
422	VALIDATION_FAILED	Body, query or parameter values fail schema validation. details lists fields.
429	RATE_LIMITED	Rate limit exceeded. Retry-After header included.
500	INTERNAL	Unexpected error. Logged with request id; client sees a generic message.

7.8 Realistic error examples

Validation failure (422) - POST /projects/:projectId/tasks with an empty title and unknown priority

```
{ "error": { "code": "VALIDATION_FAILED", "message": "Request validation failed",
    "details": [ { "path": "title", "message": "Too small: expected string to have >=1 characters" },
                 { "path": "priority", "message": "Invalid option: expected one of
                     low|medium|high|urgent" } ],
    "requestId": "7f3c1d52-3b8e-4c07-9a2e-5d1b0f6a8c11" } }
```

Unauthorized (401) - missing or expired token

```
{ "error": { "code": "TOKEN_EXPIRED", "message": "Authentication required", "requestId": "0b9e..." } }
```

Forbidden (403) - a Viewer tries to create a task

```
{ "error": { "code": "FORBIDDEN", "message": "You do not have permission to do this", "requestId":
    "c41a..." } }
```

Not found (404) - id from another organization

```
{ "error": { "code": "NOT_FOUND", "message": "Task not found", "requestId": "92de..." } }
```

Server error (500) - unexpected failure; details only in logs

```
{ "error": { "code": "INTERNAL", "message": "Something went wrong", "requestId": "ae07..." } }
```

7.9 Logging without leaking secrets

```
// config/logger.js
export const logger = pino({
  level: env.LOG_LEVEL,
  redact: { paths: ['req.headers.authorization', 'req.headers.cookie', 'res.headers["set-cookie"]',
                    '*.password', '*.newPassword', '*.currentPassword', '*.passwordHash', '*.token',
                        '*.accessToken'],
            censor: '[redacted]' },
});
```

- Log method, path (without query strings that may carry tokens), status, duration, user id and request id.
- Log security-relevant events at warn level: failed logins, refresh-token reuse, forbidden attempts, rate-limit hits.
- Never log request bodies for auth routes. Treat logs as sensitive and restrict who can read them.

7.10 Security checklist

Control	Implementation	Verified by
Passwords hashed with bcrypt, cost 12	auth.service	Unit test: hash differs from password, compare works
Password length 10 to 72 bytes enforced	Zod schema	Integration test: 73-byte password rejected
Generic login error, equalized timing	auth.service	Test: unknown email and wrong password give identical responses
JWT algorithm pinned, issuer and audience checked	authenticate	Test: token with alg none rejected
Short access token lifetime, refresh rotation, reuse detection	auth.service	Test: reused refresh token revokes the family
httpOnly, Secure, SameSite=Strict refresh cookie	auth.controller	Test: Set-Cookie attributes
tokenVersion check on every request	authenticate	Test: old token rejected after password change
Authorization on server for every route	authorize + services	Permission matrix test (Section 10)
Tenant scope on all queries	scope helper	Cross-tenant test suite
IDOR returns 404	loadContext	Cross-project tests
Strict input validation	Zod schemas	Validation tests for each endpoint
NoSQL operator injection blocked	Zod + sanitizeFilter	Test: object in email field returns 422
Mass assignment prevented	Allow-listed update fields	Test: PATCH with role or organization ignored or rejected
Rate limiting and body size cap	express-rate-limit, json limit	Test: 11th login attempt returns 429
Secure headers and CORS allowlist	helmet, cors	Test: disallowed origin gets no CORS headers
Secrets only in environment, validated on startup	config/env.js	App fails to start with missing JWT secret
No secrets or tokens in logs	pino redact	Test: log output scanned for known secret strings
Activity log append-only	Schema hooks, no routes	Test: update and delete attempts throw
Dependency hygiene	npm audit, Dependabot	CI step
HTTPS everywhere in production	Host configuration, HSTS	Deployment checklist (Section 12)

7.11 Principal security risks and trade-offs

Risk	Mitigation	Residual risk accepted in the MVP
Stolen access token	15 minute lifetime, memory-only storage	Usable until expiry unless tokenVersion changes
XSS in the SPA	React escaping, no raw HTML, CSP on hosting, httpOnly refresh cookie	An XSS bug could still act as the user while the page is open
Credential stuffing	Rate limits, bcrypt cost, generic errors	No lockout or 2FA yet
Account enumeration via registration	Rate limiting	409 reveals registered emails
Free-tier cold starts and limits	Health check, documented expectations	Slow first request after idle
Missing audit if event handler fails	Errors logged and alerted	Rare missing activity row (Section 9.7)

8. REST API Design

8.1 Conventions

Topic	Convention
Base URL	/api/v1. All paths in this section are relative to it.
Format	JSON in and out (Content-Type: application/json). Dates are ISO 8601 UTC strings. Ids are 24-character hexadecimal strings.
Authentication	Authorization: Bearer ACCESS_TOKEN on every endpoint except those marked Public or Cookie.
Success envelope	`{ "data": <object or array> }`. Lists add `"meta": { "page", "limit", "total", "totalPages" }`, or `"meta": { "nextCursor" }` for cursor-paginated feeds.
Error envelope	`{ "error": { "code", "message", "details?", "requestId" } }` (Section 7.7).
Pagination	page (default 1) and limit (default 20, maximum 100). Activity and notifications use cursor and limit.
Sorting	`sort=-createdAt,priority`: comma-separated, minus for descending, fields from an allow-list per resource.
Filtering	Plain query parameters (status, assignee, priority, dueBefore, dueAfter, q). Unknown parameters are rejected with 422.
Updates	PATCH with a partial body. Only allow-listed fields are changeable.
Common errors	Every authenticated endpoint may return 401 and 429. Every endpoint with a body or query may return 422. Endpoints with resource ids may return 400 and 404. These are omitted from the tables unless special.
Documentation	The same contract is maintained in openapi.yaml and served with Swagger UI at /api/docs (not in production, or behind authentication).

8.2 Endpoint reference

Access column: **Public**; **Cookie** (refresh cookie); **Auth** (any signed-in user); or a permission key from Section 6.12 with the roles that hold it. Input lists body fields unless marked *q:* (query string).

8.2.1 Authentication

Endpoint	Purpose	Access	Input	Success / errors
POST `/auth/register`	Create account and first organization	Public	name, email, password, organizationName?	201 {user, accessToken}; 409, 422
POST `/auth/login`	Log in	Public	email, password	200 {accessToken, user} + cookie; 401 INVALID_CREDENTIALS
POST `/auth/refresh`	Rotate refresh token, issue access token	Cookie	none	200 {accessToken} + cookie; 401
POST `/auth/logout`	Revoke current refresh token	Cookie	none	204
POST `/auth/logout-all`	Revoke all sessions	Auth	none	204

8.2.2 Users

Endpoint	Purpose	Access	Input	Success / errors
GET `/users/me`	Current user and organization memberships	Auth	none	200 {user, memberships[]}
PATCH `/users/me`	Update profile	Auth	name	200 user
PUT `/users/me/password`	Change password (ends other sessions)	Auth	currentPassword, newPassword	204; 422 if current password wrong
DELETE `/users/me`	Delete (anonymize) account	Auth	password	204; 409 sole Owner of an organization

8.2.3 Teams (organizations)

Endpoint	Purpose	Access	Input	Success / errors
`GET /organizations`	List my organizations	Auth	q: page, limit	200 list with my role
`POST /organizations`	Create organization; caller becomes Owner	Auth	name	201 organization
`GET /organizations/:orgId`	Organization details	Any member	none	200; 404
`PATCH /organizations/:orgId`	Update settings	org:update (Owner, Admin)	name	200; 403

8.2.4 Members, invitations and permissions

Endpoint	Purpose	Access	Input	Success / errors
`GET /organizations/:orgId/members`	List members with roles	Any member	q: role, q, page, limit	200 list
`PATCH /organizations/:orgId/members/:userId`	Change a member's role	member:role (Owner; Admin limited)	role	200; 403; 409 last Owner
`DELETE /organizations/:orgId/members/:userId`	Remove member; unassign their open work	member:remove	none	204; 403; 409
`POST /organizations/:orgId/invitations`	Create invitation link	member:invite (Owner, Admin)	email, role, projectIds?	201 {invitation, acceptUrl}; 409 already a member
`GET /organizations/:orgId/invitations`	List invitations	member:invite	q: status, page	200 list
`DELETE /organizations/:orgId/invitations/:invitationId`	Revoke invitation	member:invite	none	204
`POST /invitations/accept`	Accept invitation (email must match)	Auth	token	200 {organization, role}; 404 invalid or expired; 403 email mismatch

8.2.5 Projects and project members

Endpoint	Purpose	Access	Input	Success / errors
`POST /organizations/:orgId/projects`	Create project	project:create (Owner, Admin)	name, key, description?	201 project; 409 key exists
`GET /organizations/:orgId/projects`	List accessible projects	Any member	q: status, q, sort, page, limit	200 list
`GET /projects/:projectId`	Project details	project:read	none	200; 404
`PATCH /projects/:projectId`	Edit name or description	project:update	name?, description?	200; 403
`POST /projects/:projectId/archive`	Archive (read-only)	project:archive	none	200
`POST /projects/:projectId/unarchive`	Restore	project:archive	none	200
`DELETE /projects/:projectId`	Soft delete	project:delete (Owner)	none	204; 403
`GET /projects/:projectId/members`	List project members	project:read	q: page, limit	200 list
`POST /projects/:projectId/members`	Add a project member	projectMember:manage	userId	201; 409; 422 user not in organization
`DELETE /projects/:projectId/members/:userId`	Remove; their open work is unassigned	projectMember:manage	none	204

8.2.6 Tasks

Endpoint	Purpose	Access	Input	Success / errors
`POST /projects/:projectId/tasks`	Create task	task:create	title, description?, assignee?, dueDate?, priority?, labels?, status?	201 task; 403
`GET /projects/:projectId/tasks`	List tasks	project:read	q: status, assignee, priority, dueBefore, dueAfter, q, sort, page, limit	200 list
`GET /tasks/:taskId`	Task details	project:read	none	200; 404
`PATCH /tasks/:taskId`	Edit fields	task:update	title?, description?, assignee?, dueDate?, priority?, labels?	200; 403; 422
`PATCH /tasks/:taskId/status`	Move card / change status	task:move	status, position?	200 task
`DELETE /tasks/:taskId`	Delete task and its comments	task:delete (Member: own)	none	204; 403

8.2.7 Issues

Endpoint	Purpose	Access	Input	Success / errors
`POST /projects/:projectId/issues`	Report issue	issue:create	title, severity, description?, stepsToReproduce?, expectedBehavior?, actualBehavior?, priority?, assignee?	201 issue; 403; 422
`GET /projects/:projectId/issues`	List issues	project:read	q: status, severity, assignee, priority, q, sort, page, limit	200 list
`GET /issues/:issueId`	Issue details	project:read	none	200; 404
`PATCH /issues/:issueId`	Edit fields	issue:update	any editable field	200; 403
`PATCH /issues/:issueId/status`	Change status / move	issue:move	status, position?	200 issue
`DELETE /issues/:issueId`	Delete issue and its comments	issue:delete (Member: own)	none	204; 403
`GET /projects/:projectId/board`	Kanban data: tasks and issues grouped by status	project:read	q: perColumn (default 25), assignee?, type?	200 {columns}

8.2.8 Comments

Endpoint	Purpose	Access	Input	Success / errors
`GET /tasks/:taskId/comments` `GET /issues/:issueId/comments`	List comments (oldest first)	project:read	q: page, limit	200 list
`POST /tasks/:taskId/comments` `POST /issues/:issueId/comments`	Add comment	comment:create	body	201 comment; 403
`PATCH /comments/:commentId`	Edit own comment	comment:update: own	body	200; 403
`DELETE /comments/:commentId`	Delete comment	comment:delete (Member: own)	none	204; 403

8.2.9 Notifications

Endpoint	Purpose	Access	Input	Success / errors
`GET /notifications`	My notifications, newest first	Auth	q: unread?, cursor, limit	200 list + nextCursor
`GET /notifications/unread-count`	Unread count (polled)	Auth	none	200 {count}
`PATCH /notifications/:notificationId/read`	Mark one as read	Auth (own)	none	200; 404
`POST /notifications/read-all`	Mark all as read	Auth	none	204

8.2.10 Activity logs and dashboard analytics

Endpoint	Purpose	Access	Input	Success / errors
`GET /projects/:projectId/activity`	Project timeline	activity:project	q: action, actor, cursor, limit	200 list + nextCursor
`GET /organizations/:orgId/activity`	Organization-wide timeline	activity:org (Owner, Admin)	q: projectId?, action, actor, cursor, limit	200; 403
`GET /projects/:projectId/analytics`	Completion, counts by status, overdue, workload	analytics:read	none	200 {summary}
`GET /organizations/:orgId/dashboard`	Progress of each accessible project, overdue totals	Any member	none	200 {projects[], totals}

This reference contains exactly the endpoints required by the defined feature set (57 endpoints). No endpoints exist for features outside the MVP scope, such as password reset, file upload or organization deletion. The only extra route is the operational GET /healthz used by the host (Section 12).

8.3 Request and response examples

POST /auth/register

```
Request:  { "name": "Asha Rai", "email": "asha@example.com", "password": "correct-horse-battery",
    "organizationName": "Rai Studio" }
201:      { "data": { "user": { "id": "665f1c2e9a1b4c0012ab0001", "name": "Asha Rai", "email":
    "asha@example.com" },
                    "accessToken": "eyJhbGciOiJIUzI1NiIs..." } }            + Set-Cookie: rt=...;
                       HttpOnly; Secure; SameSite=Strict; Path=/api/v1/auth
409:      { "error": { "code": "CONFLICT", "message": "An account with this email already exists",
    "requestId": "..." } }
422:      { "error": { "code": "VALIDATION_FAILED", "details": [ { "path": "password", "message": "Must
    be 10 to 72 bytes" } ], ... } }
```

POST /auth/login

```
Request:  { "email": "asha@example.com", "password": "correct-horse-battery" }
200:      { "data": { "accessToken": "eyJ...", "user": { "id": "665f1c2e9a1b4c0012ab0001", "name": "Asha
    Rai", "email": "asha@example.com" } } }
401:      { "error": { "code": "INVALID_CREDENTIALS", "message": "Invalid email or password",
    "requestId": "..." } }
429:      { "error": { "code": "RATE_LIMITED", "message": "Too many attempts, try again later",
    "requestId": "..." } }   + Retry-After: 540
```

POST /organizations/:orgId/invitations

```
Request:  { "email": "dev@example.com", "role": "member", "projectIds": ["665f1c2e9a1b4c0012ab0003"] }
201:      { "data": { "invitation": { "id": "665f...0010", "email": "dev@example.com", "role": "member",
    "status": "pending",
                                  "expiresAt": "2026-11-10T09:00:00.000Z" },
                    "acceptUrl": "https://app.example.com/invite/9Qx...one-time-token" } }
409:      { "error": { "code": "CONFLICT", "message": "This person is already a member", "requestId":
    "..." } }
```

POST /projects/:projectId/tasks

```
Request:  { "title": "Implement login form", "priority": "high", "assignee": "665f1c2e9a1b4c0012ab0001",
    "dueDate": "2026-11-20", "labels": ["auth", "frontend"] }
201:      { "data": { "id": "665f1c2e9a1b4c0012ab0004", "key": "ECOM-12", "number": 12, "title":
    "Implement login form", "status": "backlog",
                    "position": 4096, "priority": "high", "assignee": "665f1c2e9a1b4c0012ab0001",
                        "dueDate": "2026-11-20T00:00:00.000Z",
                    "labels": ["auth", "frontend"], "project": "665f1c2e9a1b4c0012ab0003", "createdAt":
                        "2026-11-03T08:30:00.000Z" } }
403:      { "error": { "code": "FORBIDDEN", "message": "You do not have permission to do this",
    "requestId": "..." } }
```

GET /projects/:projectId/tasks?status=todo&sort;=-priority&page;=1&limit;=2

```
200: { "data": [ { "id": "665f1c2e9a1b4c0012ab0004", "key": "ECOM-12", "title": "Implement login form",
    "status": "todo", "priority": "high" },
                { "id": "665f1c2e9a1b4c0012ab0005", "key": "ECOM-9", "title": "Product list page",
                    "status": "todo", "priority": "medium" } ],
       "meta": { "page": 1, "limit": 2, "total": 7, "totalPages": 4 } }
```

PATCH /tasks/:taskId/status

```
Request:  { "status": "in_review", "position": 2048 }
200:      { "data": { "id": "665f1c2e9a1b4c0012ab0004", "key": "ECOM-12", "status": "in_review",
    "position": 2048 } }
422:      { "error": { "code": "VALIDATION_FAILED", "details": [ { "path": "status", "message": "Invalid
    option" } ], ... } }
```

POST /projects/:projectId/issues

```
Request:  { "title": "Login fails with plus sign in email", "severity": "major", "priority": "high",
            "stepsToReproduce": "1. Open /login 2. Enter a+b@example.com 3. Submit",
            "expectedBehavior": "User is logged in", "actualBehavior": "Server returns 422" }
201:      { "data": { "id": "665f1c2e9a1b4c0012ab0020", "key": "ECOM-13", "title": "Login fails with plus
    sign in email",
                    "severity": "major", "status": "backlog", "createdBy": "665f1c2e9a1b4c0012ab0001" } }
```

POST /tasks/:taskId/comments

```
Request:  { "body": "Added validation. Ready for review." }
201:      { "data": { "id": "665f1c2e9a1b4c0012ab0030", "author": "665f1c2e9a1b4c0012ab0001", "body":
    "Added validation. Ready for review.",
                    "createdAt": "2026-11-03T09:10:00.000Z" } }
```

GET /notifications/unread-count and GET /notifications?unread=true&limit;=1

```
200: { "data": { "count": 3 } }
200: { "data": [ { "id": "665f1c2e9a1b4c0012ab0040", "type": "task_assigned", "message": "Asha assigned
    you ECOM-12",
                "targetType": "task", "target": "665f1c2e9a1b4c0012ab0004", "readAt": null,
                    "createdAt": "2026-11-03T08:30:01.000Z" } ],
       "meta": { "nextCursor": "MjAyNi0xMS0wM1QwODozMDowMS4wMDBafDY2NWY..." } }
```

GET /projects/:projectId/activity?limit=1

```
200: { "data": [ { "id": "665f1c2e9a1b4c0012ab0050", "action": "task.status_changed", "actor": { "id":
    "665f1c2e9a1b4c0012ab0001", "name": "Asha Rai" },
                "resourceType": "task", "resourceId": "665f1c2e9a1b4c0012ab0004", "summary": "ECOM-12
                    Implement login form",
                "changes": { "before": { "status": "in_progress" }, "after": { "status": "in_review" }
                    },
                "createdAt": "2026-11-03T09:05:00.000Z" } ], "meta": { "nextCursor": null } }
```

GET /projects/:projectId/analytics

```
200: { "data": { "totals": { "items": 42, "done": 18, "completionRate": 0.43, "overdue": 3 },
                 "byStatus": { "backlog": 8, "todo": 9, "in_progress": 5, "in_review": 2, "done": 18 },
                 "workload": [ { "user": "665f1c2e9a1b4c0012ab0001", "name": "Asha Rai", "open": 6,
                     "overdue": 1 } ] } }
```

9. Activity Logs and Notifications

9.1 How events flow

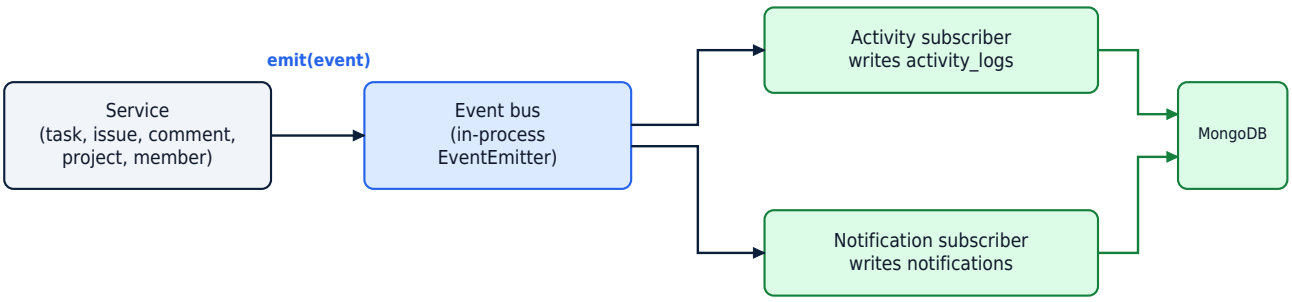


Figure 7. Services publish domain events; subscribers create activity records and notifications.

A service performs its primary write, then emits an event containing the context, the resource and the before/after values of whitelisted fields. Two subscribers react: one writes the activity record, the other works out who should be notified. Services do not know the subscribers exist, so adding a new reaction (for example email) does not change business code.

9.2 Events that must be recorded

Action name	Trigger	Changes recorded (before and after)
organization.updated	Settings edited	name
member.invited, invitation.revoked, member.joined	Invitation lifecycle	email, role (never the token)
member.role_changed, member.removed	Role or membership change	role
project.created, updated, archived, unarchived, deleted	Project lifecycle	name, status
project.member_added, project.member_removed	Project membership	user id
task.created, task.deleted, issue.created, issue.deleted	Lifecycle	key and title in summary
task.updated, issue.updated	Field edits	title, priority, dueDate, labels (changed fields only)
task.status_changed, issue.status_changed	Kanban move or status edit	status
task.assigned, issue.assigned	Assignee change	assignee
comment.created, comment.deleted	Comments	target reference in summary; no body stored

Not recorded in activity logs: reads, logins and password changes. Security events such as login failures go to the application log (Section 7.9). Comment text and passwords are never copied into the activity log.

9.3 Activity record

```
{ "organization": "665f1c2e9a1b4c0012ab0002", "project": "665f1c2e9a1b4c0012ab0003", "actor":
    "665f1c2e9a1b4c0012ab0001", "action": "task.assigned",
  "resourceType": "task", "resourceId": "665f1c2e9a1b4c0012ab0004", "summary": "ECOM-12 Implement login
      form",
  "changes": { "before": { "assignee": null }, "after": { "assignee": "665f1c2e9a1b4c0012ab0007" } },
  "requestId": "7f3c1d52-3b8e-4c07-9a2e-5d1b0f6a8c11", "createdAt": "2026-11-03T08:30:00.000Z" }
```

9.4 Protecting logs from modification

- No endpoint creates, updates or deletes activity logs directly. Only activity.service.record() inserts them, from event subscribers.
- The model rejects mutation at the schema level:

```
// modules/activity/activity.model.js
const BLOCKED = ['updateOne', 'updateMany', 'findOneAndUpdate', 'replaceOne', 'deleteOne', 'deleteMany',
```

```
    'findOneAndDelete'];
BLOCKED.forEach((op) => schema.pre(op, () => { throw new Error('activity_logs is append-only'); }));
schema.pre('save', function () { if (!this.isNew) throw new Error('activity_logs is append-only'); });
```

- Reading is restricted by RBAC and tenant scope: project timeline for project members, organization timeline for Owners and Admins.
- Where the database tier allows custom roles, give the application's database user insert and find permissions only on activity_logs, so even a code bug cannot rewrite history.
- Logs are included in database backups. Later: hash chaining to make tampering evident.

9.5 Notification creation rules

Event	Recipients	Example message
Task or issue assigned	The new assignee	Asha assigned you ECOM-12
Status changed	Assignee and creator	ECOM-12 moved to In Review
Comment added	Assignee and creator, and previous commenters on that item	Asha commented on ECOM-12
Added to or removed from a project	The affected user	You were added to E-commerce Platform
Invitation accepted	The inviter	Dev joined Rai Studio

- The actor is always removed from the recipient list.
- Recipients are de-duplicated and must still have access to the project at creation time.
- Notification text is generated from a template and truncated to 200 characters; it contains no secrets or full comment bodies.

9.6 Unread counts and read status

- Unread means readAt is null. The count is countDocuments({ recipient, readAt: null }), served by the index (recipient, readAt, createdAt).
- The client polls GET /notifications/unread-count every 30 to 60 seconds while the tab is visible and refreshes immediately after the user's own actions.
- Mark one: set readAt if the notification belongs to the caller (otherwise 404). Mark all: updateMany filtered by recipient and readAt null.
- The feed uses cursor pagination (createdAt, _id) and a TTL index removes old notifications.
- Later: Server-Sent Events or WebSockets for real-time delivery, plus per-user preferences and email digests.

9.7 Failure handling and its trade-off

In the MVP, subscribers run in-process and are awaited so that tests are deterministic. A failure in a subscriber is caught, logged with the request id and does not roll back the user's action. The trade-off is a rare missing activity row or notification after an internal error. The hardening path, if accountability requirements grow, is to write the activity record inside the same database transaction as the primary change, or to use a transactional outbox collection processed by a worker.

10. Testing and Quality Assurance

10.1 Strategy

Level	What it covers	Tools	Share of effort
Unit	Pure logic: permission map, position midpoint calculation, token helpers, validators, notification recipient rules	Vitest	About 30%
Integration (API)	Full HTTP stack against a real in-memory MongoDB: auth flows, CRUD, validation, error shapes, activity and notification side effects	Vitest, Supertest, mongodb-memory-server	About 45%
Authorization	Permission matrix and tenant isolation, generated from tables so every role and endpoint is covered	Vitest, Supertest	About 25%
Frontend (optional)	Critical components: login form, Kanban move with rollback, route guard	Vitest, React Testing Library	Added after the API is stable
End-to-end (later)	One happy path through the browser	Playwright	Post-MVP

10.2 Test environment

- app.js exports the Express app without listening, so Supertest can call it directly.
- A global setup starts mongodb-memory-server (configured as a replica set when transactions are tested), connects Mongoose and clears collections between tests.
- Factories create users, organizations, memberships, projects and tasks. A helper loginAs(role) returns an access token for a user with that role in a given project.
- Tests never use production data or secrets. NODE_ENV=test uses a fixed test secret and a low bcrypt cost (4) to keep tests fast.

```
// tests/integration/crossProject.test.js
import request from 'supertest';
import { app } from '../../src/app.js';
import { seedTwoProjects } from '../helpers/seed.js';
describe('cross-project access', () => {
  it('returns 404 when a Member of project A reads a task of project B', async () => {
    const { memberA, taskB } = await seedTwoProjects();
    const res = await request(app).get(`/api/v1/tasks/${taskB.id}`).set('Authorization', `Bearer
        ${memberA.token}`);
    expect(res.status).toBe(404);
    expect(res.body.error.code).toBe('NOT_FOUND');
  });
});
// tests/authorization/matrix.test.js - generated from the permission matrix
describe.each(PERMISSION_CASES)('%s as %s', (action, role, expectedStatus) => {
  it(`responds ${expectedStatus}`, async () => {
    const res = await performAs(role, action);
    expect(res.status).toBe(expectedStatus);
  });
});
```

10.3 Test matrix

Priority: **P0** must pass before any deployment; **P1** must pass before a release is called complete; **P2** desirable. Type: U = unit, I = integration, A = authorization.

ID	Scenario	Expected result	Pri	Type
AUTH-01	Register with valid data	201, user and organization created, owner membership exists, no passwordHash in response	P0	I
AUTH-02	Register with duplicate email (different case)	409 CONFLICT	P0	I
AUTH-03	Register with weak or 73-byte password, invalid email	422 with field details	P0	I
AUTH-04	Login with valid credentials	200, access token, refresh cookie with HttpOnly, Secure, SameSite=Strict	P0	I

ID	Scenario	Expected result	Pri	Type
AUTH-05	Login with wrong password and with unknown email	Both 401 INVALID_CREDENTIALS with identical body	P0	I
AUTH-06	Login with object payload {email: {$gt: ''}}	422, no query executed	P0	I
AUTH-07	11th login attempt inside the window	429 with Retry-After	P1	I
AUTH-08	Request with expired access token	401 TOKEN_EXPIRED	P0	I
AUTH-09	Request with tampered token or alg none	401 INVALID_TOKEN	P0	I
AUTH-10	Refresh with valid cookie	200 new access token, old refresh token revoked	P0	I
AUTH-11	Reuse of an already rotated refresh token	401 and the whole token family revoked	P0	I
AUTH-12	Logout, then refresh with the old cookie	401	P0	I
AUTH-13	Change password, then use an older access token	401 (tokenVersion mismatch)	P0	I
ACC-01	Protected endpoint without Authorization header	401 MISSING_TOKEN	P0	I
RBAC-01	Each role against each permission-matrix row	Allowed roles succeed, others get 403	P0	A
RBAC-02	Admin changes an Owner's role, or promotes someone to Admin	403	P0	A
RBAC-03	Remove or demote the last Owner	409	P0	I
RBAC-04	Viewer attempts create, edit, move, comment, delete	403 for each	P0	A
TEN-01	User from organization B requests any organization A resource (every endpoint)	404, no data leaked	P0	A
TEN-02	Member of project A reads, edits or comments on project B item	404	P0	A
TEN-03	Assign a task to a user who is not a project member or is in another organization	422 or 404, task unchanged	P0	I
TEN-04	Mass assignment: PATCH with organization, createdBy or role fields	Rejected or ignored, stored values unchanged	P0	I
PRJ-01	Create project with duplicate key in the same organization	409	P1	I
PRJ-02	Archive then create a task in the archived project	409 or 403, project read-only	P1	I
PRJ-03	Soft-deleted project and its tasks	404 on every lookup	P1	I
TSK-01	Create task with valid data	201, number assigned from project counter, position set	P0	I
TSK-02	Two simultaneous task creations in one project	Distinct sequential numbers	P1	I
TSK-03	Create task with invalid status, empty title, 11 labels	422 with details	P0	I
TSK-04	Move task to another status and position	200, status and position updated, completedAt set when Done	P0	I
TSK-05	Member deletes another user's task; deletes own task	403; 204 and comments removed	P0	A
TSK-06	Overdue calculation for tasks past due and not Done	Counted in analytics overdue	P1	I
CMT-01	Edit another user's comment; delete as Member and as Admin	403; Member own only, Admin any	P1	A
ACT-01	Create, update, move, assign, delete a task	One activity row per action with actor, action, resource and diff	P0	I
ACT-02	Attempt to update or delete an activity row	Throws append-only error	P0	U
ACT-03	Activity rows never contain passwords, tokens or comment bodies	Field scan passes	P1	I
NTF-01	Assign task to another user	Notification for assignee only, none for the actor	P1	I

ID	Scenario	Expected result	Pri	Type
NTF-02	Unread count, mark one read, mark all read, another user's notification id	Counts correct; foreign id returns 404	P1	I
ERR-01	Unhandled exception inside a service	500 INTERNAL with requestId, no stack trace in body	P0	I
ERR-02	Malformed id in URL; malformed JSON	400 BAD_REQUEST	P1	I
DB-01	Transaction rollback when creating organization fails midway	No partial user, organization or membership	P1	I
DB-02	Duplicate membership insert race	Unique index rejects second insert	P1	I
UNIT-01	Position midpoint and rebalance logic	Order preserved, no collisions	P1	U
UNIT-02	can(role, permission) for every role	Matches the permission matrix	P0	U

10.4 Running the tests

```
// server/package.json (scripts, with "type": "module")
"scripts": {
  "dev": "nodemon src/server.js",
  "start": "node src/server.js",
  "lint": "eslint .",
  "test": "vitest run",
  "test:unit": "vitest run tests/unit",
  "test:int": "vitest run tests/integration tests/authorization",
  "test:watch": "vitest",
  "test:coverage": "vitest run --coverage"
}
# run everything once, as CI does
cd server && npm test
```

10.5 Gate before deployment

- All P0 and P1 tests pass in CI on the exact commit to be deployed.
- Lint passes and there are no high or critical vulnerabilities in production dependencies (npm audit --omit=dev).
- The index creation script runs cleanly against a fresh database.
- A manual smoke test passes on staging or production: register, log in, create project, create task, move card, comment, check notification and activity, log out.
- Coverage is reviewed as a signal, not a target. A reasonable aim is high coverage on services and middleware, with the permission matrix at 100% of rows.

```
# .github/workflows/ci.yml (outline)
on: [push, pull_request]
jobs:
  server:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: 22, cache: npm, cache-dependency-path: server/package-lock.json }
      - run: cd server && npm ci && npm run lint && npm test
```

11. Development Roadmap

Phases are sequential; each ends with working, committed, tested software. Size is relative (S, M, L) so the plan stays honest before real velocity is known: measure Phase 1 and Phase 2, then re-estimate the rest.

Phase	Deliverables	Depends on	Completion criteria
1. Foundation and architecture Size S	Monorepo with client and server; Vite + Tailwind setup; Express app skeleton (app.js, server.js); env validation; logger; error classes and error handler; request id; health route; ESLint and Prettier; CI running an empty test suite; architecture diagram committed to /docs.	None	App boots locally; GET /healthz returns 200; CI is green; invalid env makes the server refuse to start.
2. Database schemas and authentication Size M	Mongoose models for all collections and indexes; register, login, refresh, logout, logout-all; bcrypt; JWT middleware; users/me endpoints; React login and register pages, AuthProvider, ProtectedRoute, apiClient with refresh.	Phase 1	AUTH-01 to AUTH-13 and ACC-01 pass; a user can sign up, reload the page and stay signed in; passwordHash never appears in any response.
3. RBAC and team management Size M	Organizations, memberships, invitations (link-based), role changes and removal; permission map; loadContext and authorize middleware; members and invitations UI.	Phase 2	RBAC-01 to RBAC-04 and TEN-01 pass; last-Owner rule enforced; invited user can join through the link.
4. Projects, tasks and issue tracking Size L	Projects and project members; task and issue CRUD with filters, sorting, pagination; project counter numbering; soft delete; list pages and forms; Swagger spec for these modules.	Phase 3	PRJ, TSK and TEN-02 to TEN-04 tests pass; every list endpoint paginates; OpenAPI matches behavior.
5. Kanban, comments and notifications Size L	Board endpoint; drag-and-drop with optimistic update and rollback; position handling; comments; notification subscribers, unread count polling and notification UI.	Phase 4	TSK-04, CMT-01, NTF-01 and NTF-02 pass; moving a card persists after reload; a failed move rolls back visibly.
6. Activity logs and analytics Size M	Event bus and activity subscriber; append-only enforcement; project and organization timelines; analytics aggregations; dashboard UI.	Phase 5	ACT-01 to ACT-03 pass; dashboard numbers match a hand-counted sample; analytics queries use indexes (checked with explain).
7. Security hardening and automated testing Size M	Rate limiting, helmet, CORS allowlist, sanitizeFilter, mass-assignment tests, log redaction, remaining test-matrix rows, dependency audit, security checklist review.	Phases 2 to 6	All P0 and P1 matrix rows pass; checklist in Section 7.10 fully verified; no high or critical audit findings.
8. Deployment, documentation and portfolio presentation Size S	Deploy to Vercel, Render and Atlas; same-origin proxy; seed script with a demo organization; README with screenshots, architecture diagrams and the real problems solved; Swagger docs; short demo video.	Phase 7	Deployment checklist (Section 12) complete; a stranger can run the demo from the README; documentation distinguishes what is implemented from what is planned.

Definition of done for any task in any phase: code merged by pull request, lint and tests green, new behavior covered by a test, permissions and tenant scope considered, OpenAPI updated, no TODOs that hide a security gap.

Cut line. If time is limited, finish Phases 1 to 5 completely and treat notifications polish, analytics and invitation extras as optional. Update the README honestly to say what is built.

12. Deployment and Maintenance

12.1 Deployment topology

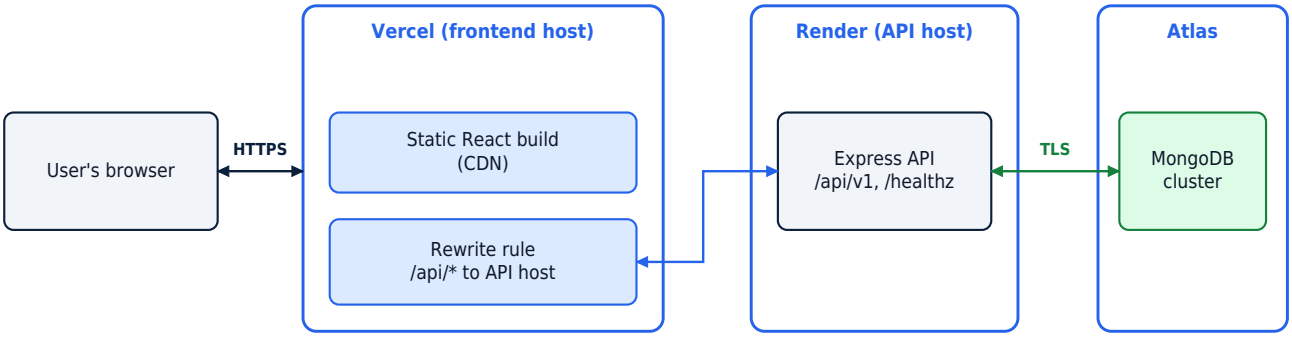


Figure 8. Production topology. The browser only talks to the frontend host; /api requests are proxied to the API.

The frontend is deployed as a static build on a CDN-backed host. A rewrite rule forwards `/api/*` to the API host, so the browser sees one origin. That keeps the refresh cookie first-party (SameSite=Strict works) and avoids third-party cookie blocking.

```
// client/vercel.json
{ "rewrites": [
    { "source": "/api/:path*", "destination": "https://YOUR-API.onrender.com/api/:path*" },
    { "source": "/(.*)",       "destination": "/index.html" }
] }
```

12.2 Frontend deployment

- Connect the GitHub repository to the frontend host; root directory client; build command npm run build; output directory dist.
- Only non-secret VITE_* variables are set. Preview deployments are created for pull requests.
- Add security headers (Content-Security-Policy, X-Content-Type-Options, Referrer-Policy) in the host configuration.

12.3 Backend deployment

- Create a web service from the repository; root directory server; build command npm ci; start command npm start.
- Set every variable from Section 4.8 in the host dashboard. NODE_ENV=production.
- Expose GET /healthz (returns 200 and a short JSON body, no database detail) and configure it as the host health check.
- Handle SIGTERM: stop accepting requests, finish in-flight ones, close the Mongoose connection.
- Free-tier instances may sleep when idle, so the first request after a pause can be slow. Document this honestly in the README; do not hide it.

12.4 MongoDB hosting

- Use a MongoDB Atlas cluster. A free shared cluster is acceptable for a portfolio MVP; check current limits before relying on it.
- Create a dedicated database user for the application with access to the ResolveHQ database only, with a long random password. Use a separate cluster or database for tests and staging.
- Network access: free hosting often has no fixed outbound IP, which may force a broad allowlist. If so, rely on TLS, strong credentials and a least-privilege user, and move to fixed IPs or private networking when upgrading.
- Run the index creation script (or rely on Mongoose autoIndex only in development) as a deliberate deployment step.

12.5 Environment configuration

Environment	Database	Frontend	Purpose
development	Local MongoDB or a personal Atlas database	Vite dev server with proxy to localhost API	Daily work
test	mongodb-memory-server	None	Automated tests
staging (optional)	Separate Atlas database	Preview deployment	Pre-release verification
production	Production Atlas database	Production deployment	Live demo

12.6 Production security

- HTTPS only; HSTS enabled. Cookies are Secure.
- All production secrets generated fresh (never reuse development values) and stored only in the host dashboard.
- Swagger UI disabled or protected in production. Debug logging off. Stack traces never returned.
- Rate limits active and trust proxy configured. CORS limited to the production origin.
- Demo accounts, if any, have low privileges and are reset by the seed script. No real personal data in the demo.

12.7 Logging, monitoring and error tracking

- Structured JSON logs to the host's log viewer, with request ids. Redaction as in Section 7.9.
- An external uptime check on /healthz, with an alert by email.
- Optional: an error tracker (for example Sentry) in both client and server, with personal data scrubbing enabled.
- Review warn-level security events (failed logins, refresh reuse, forbidden attempts) regularly.

12.8 Backups and recovery

- Check what backups the chosen database tier provides. Some free tiers provide no automated backups.
- If there is none, schedule a mongodump (for example with a scheduled GitHub Actions job) to encrypted private storage, keeping several generations.
- A backup that has never been restored is unproven. Restore into a scratch database after setup and then every quarter, and record the result.

12.9 Maintenance routine

Frequency	Task
Every pull request	CI tests and lint; review dependency changes.
Weekly	Check logs and uptime alerts; review error tracker.
Monthly	Update dependencies in a branch, run the full test matrix, run npm audit; review slow queries and indexes.
Quarterly	Test a backup restore; rotate secrets that warrant it; review the security checklist.
As needed	Remove unused data (expired invitations and tokens are cleaned by TTL indexes); revisit rate limits.

12.10 Deployment checklist

	Item
[ ]	All P0 and P1 tests green in CI on the release commit
[ ]	No high or critical npm audit findings in production dependencies
[ ]	Production environment variables set and different from development; app starts without validation errors
[ ]	JWT_ACCESS_SECRET is at least 32 random bytes and not reused anywhere
[ ]	Atlas user is least-privilege; network access reviewed; TLS connection string used
[ ]	Indexes created and checked with explain on board, overdue and activity queries
[ ]	CORS origin equals the production frontend; cookies are HttpOnly, Secure and SameSite=Strict
[ ]	Rewrite from /api to the API host works; refresh and logout work from a fresh browser profile
[ ]	/healthz configured as health check; uptime monitor created
[ ]	Swagger UI disabled or protected; stack traces absent from error responses
[ ]	Backup strategy in place and one restore tested
[ ]	Seed script creates the demo organization; no real personal data present
[ ]	Manual smoke test passed on production
[ ]	README states clearly what is implemented and what is planned

12.11 Git and GitHub workflow

Topic	Convention
Branches	main is protected and always deployable. Work happens in feature/short-name, fix/short-name and chore/short-name branches created from main.
Pull requests	Every change merges through a pull request with CI passing. Squash merge to keep main readable. Solo developers still use pull requests; they create a reviewable history.
Commit messages	Conventional Commits: type(scope): summary. Examples: feat(tasks): add status move endpoint; fix(auth): reject 73-byte passwords; test(rbac): cover viewer permissions; docs(api): update openapi.
PR checklist	Tests added or updated; permissions and tenant scope considered; OpenAPI updated; no secrets; screenshots for UI changes.
Releases	Tag each deployed version (v0.1.0, v0.2.0) and keep a short CHANGELOG.
Hotfixes	fix/ branch from main, same checks, deploy, tag a patch version.
Repository hygiene	.gitignore for .env, node_modules and build output from the first commit; secret scanning and Dependabot enabled.

13. Technical Decisions, Assumptions and Status

13.1 Technical decisions and their reasoning

Decision	Alternatives considered	Reasoning	Trade-off accepted
Modular monolith	Microservices	One deployable unit, simple local development, transactions available, clear module boundaries	Scales as one unit; extraction later requires work
Tenant = organization; roles are organization-level; project access through membership	Per-project roles; global roles	Simple mental model and permission matrix; fits small teams	Cannot make someone Admin in one project and Viewer in another (possible later)
Separate tasks and issues collections	One collection with a type field	Different fields and future workflows; clearer validation	Some duplicated code; board merges two queries
Access JWT in memory plus rotating refresh cookie	JWT in localStorage; server sessions	Limits XSS token theft, supports silent login and revocation	More moving parts than one long-lived token
Roles not stored in the JWT	Roles in token claims	Role changes and removals take effect immediately	One user lookup per request
404 for resources outside the caller's tenant	403 everywhere	Avoids revealing that an id exists	Slightly harder debugging; documented in error codes
Zod for validation	express-validator, Joi, Mongoose validators only	Single schema language, strict types block operator injection, reusable on the client	Extra dependency
In-process event bus for activity and notifications	Direct calls; message queue	Decouples modules without infrastructure	Not durable if the process dies mid-handler (Section 9.7)
Polling for unread count	WebSockets or SSE	Simple, free-tier friendly, adequate for the scale	Up to a minute of delay
Numeric position for card order	Array of ids on the board; linked list	One-document update per move, indexable	Occasional re-balance
Per-project counter with $inc for keys	Random ids only; count + 1	Readable keys (ECOM-12) and race-free numbering	Gaps after deletions
Soft delete for projects	Hard delete with cascade	Recoverable, cheap	Deleted data remains until a purge job exists
Same-origin API through a rewrite	Cross-site cookies	First-party cookie, fewer browser compatibility problems	Depends on host rewrite support
422 for validation errors	400	Distinguishes malformed requests (400) from well-formed but invalid data (422)	Some clients expect 400

13.2 Assumptions

- Node.js current LTS and Express 5. On Express 4 add an asyncHandler wrapper.
- MongoDB Atlas (a replica set), so multi-document transactions are available.
- A user can belong to several organizations; the UI shows one at a time and the organization id appears in the URL for organization-level routes.
- Invitation links are shown to the inviter and shared manually; sending email is out of scope for the MVP.
- All times are stored in UTC and displayed in the user's local time.
- No file uploads, no billing, no public API keys in the MVP.
- English-only interface for the MVP.
- Hosting limits and pricing change; verify them at deployment time.

13.3 Planned versus implemented

Area	Status
Authentication and sessions	Planned
Organizations, RBAC, invitations	Planned
Projects, tasks, issues, Kanban, comments	Planned
Notifications, activity logs, analytics	Planned
Automated tests and CI	Planned
Deployment	Planned

Update this table as phases complete and change "Planned" to "Implemented" only when the feature is merged, tested and deployed. Do not describe ResolveHQ as production-ready, and do not claim users or clients it does not have.

13.4 Open questions to settle early

- Should Members be allowed to create projects? (The matrix says no; change it in one place if the answer is yes.)
- Is link-sharing for invitations acceptable, or must email delivery exist before the first demo?
- Is a per-project role needed? If so, extend project_members with a role field and take the lower of the two roles.
- What retention period applies to notifications and invitations?

Appendix A. First setup commands (Phase 1)

```
mkdir resolvehq && cd resolvehq && git init
npm create vite@latest client -- --template react      # then add Tailwind using its current Vite guide
mkdir server && cd server && npm init -y && npm pkg set type=module
npm i express mongoose zod jsonwebtoken bcrypt helmet cors express-rate-limit cookie-parser pino
    pino-http \
      swagger-ui-express yaml dotenv
npm i -D vitest supertest mongodb-memory-server nodemon eslint prettier
# If native bcrypt fails to install on your machine, bcryptjs is a drop-in alternative (slower, pure
    JavaScript).
```

Next step after this document: Phase 1, starting with the repository structure from Section 4.3 and the environment validation from Section 4.8.